Instruction Finetuning (IF)
a.k.a.
Supervised Finetuning (SFT)

Anisio Lacerda

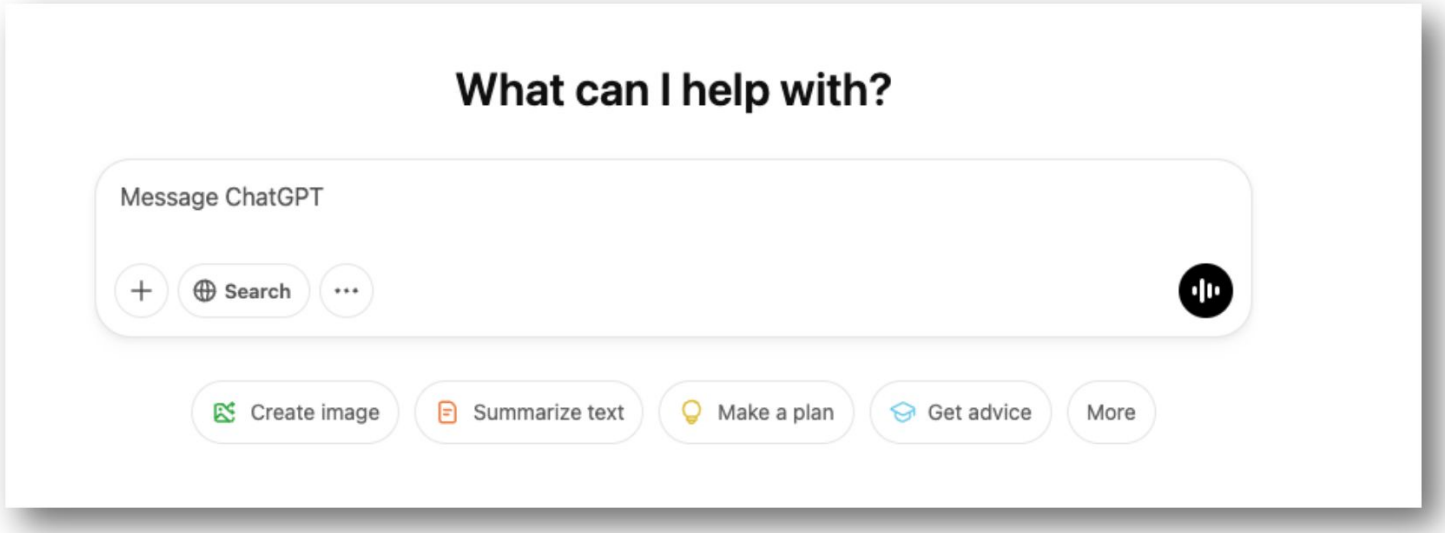
Rodrygo Santos

Language models as multitask assistants?

How do we get from this

UFMG is located in _____

to this:



Today

Previously

Classification task

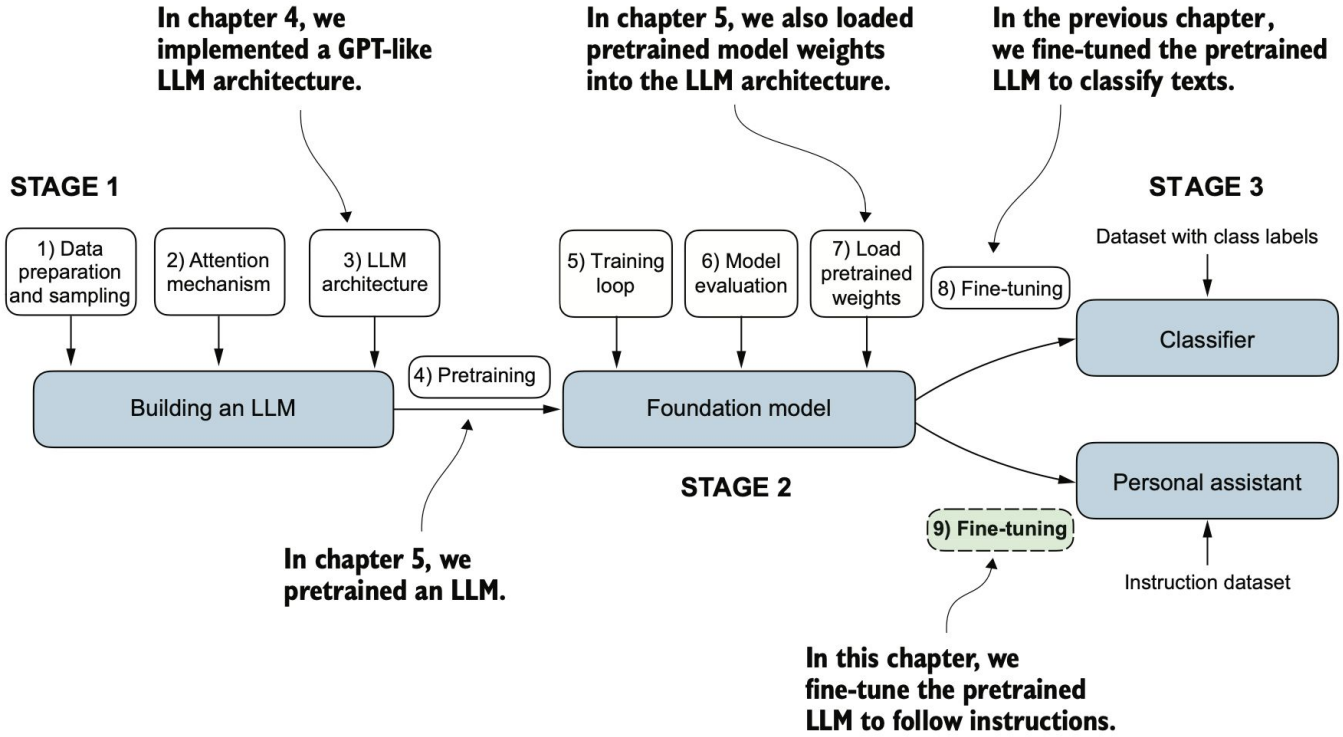


Figure 7.1 The three main stages of coding an LLM. This chapter focuses on step 9 of stage 3: fine-tuning a pretrained LLM to follow human instructions.

Today

Background and motivation

On the importance of Instruction Tuning.

Instructions

What are instructions?

Let's build from scratch

Implementing it.

Other data and models

Multimodal data and learned models.

Advanced SFT

Lora.

Background and Motivation

How do humans learn?

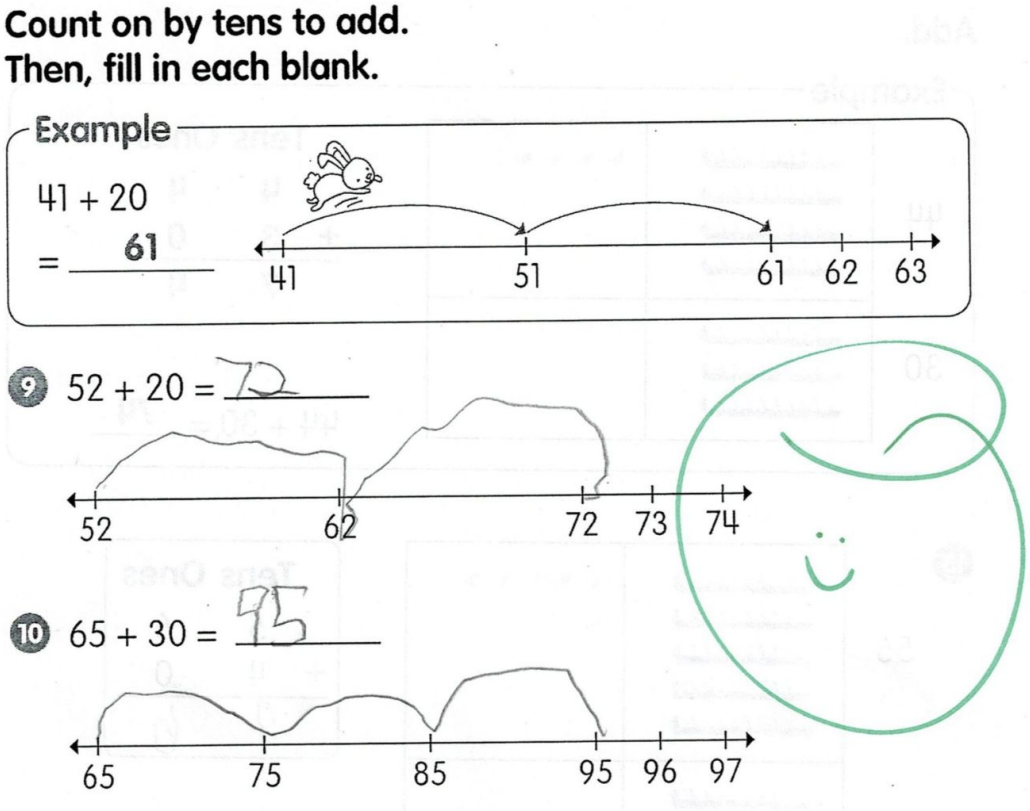
Task instruction

how to "count", and

where to "fill"

Very few examples

only one here



Typical human learning

Detailed instruction

Very few examples (<10)

How can task instructions benefit machine learning?

Mainstream machine learning

No instructions

Large training sets

Even few-shot learning often uses 100s of examples

What are "instructions"?

LLM-oriented instructions (e.g., prompts)

Human-oriented instructions (e.g., Amazon Mturk guidelines)

What are "instructions"?

LLM-oriented instructions (e.g., prompts)

Human-oriented instructions (e.g., Amazon Mturk guidelines)

How to solve this sentiment analysis problem?

This was the best pizza I've ever had!	0
You can get better sushi down the road for half the price.	1
Salmon nigiri was bad. Not worth what they're asking.	1
Excellent pizza! Slices are fantastic, prices are reasonable.	?

Pattern-Exploiting Training (PET) (Schick & Schütze, 2020/2021)

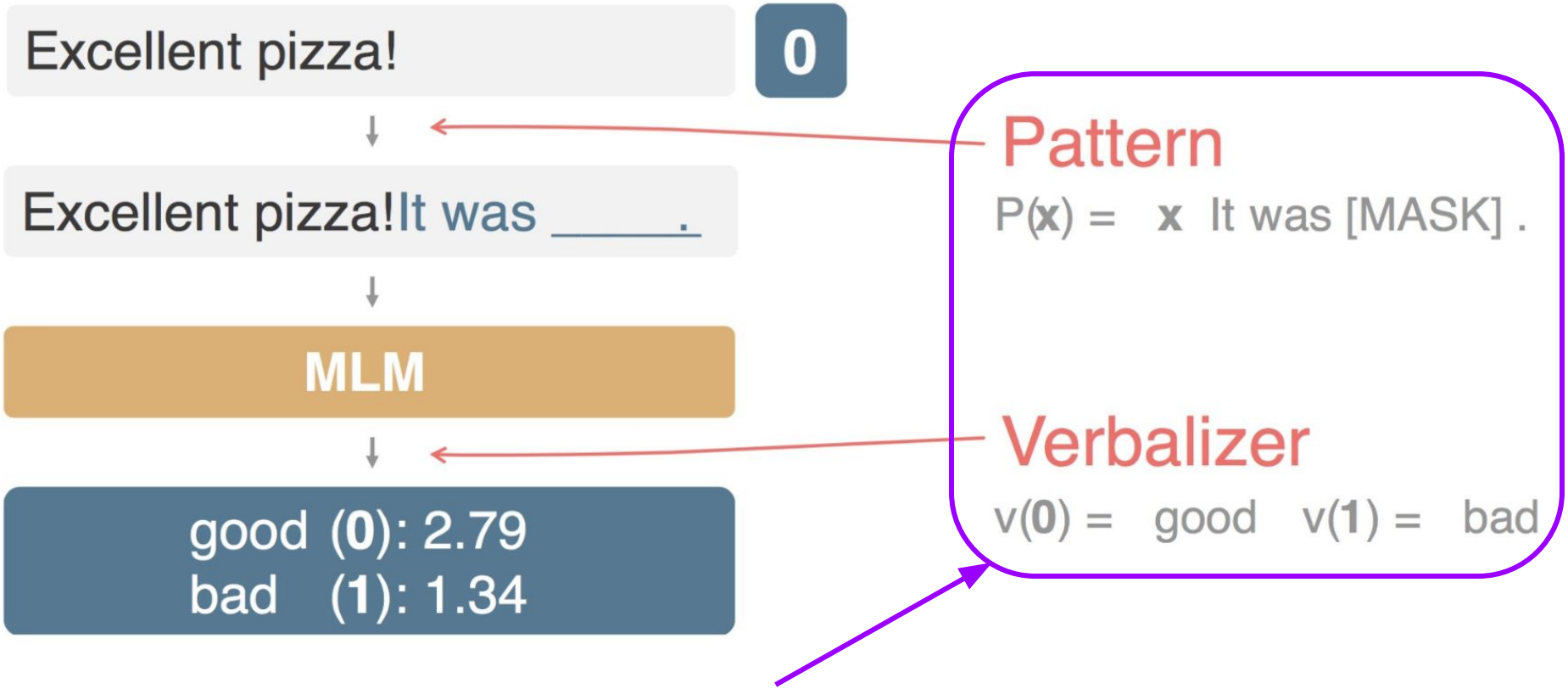
This was the best pizza I've ever had!	0
You can get better sushi down the road for half the price.	1
Salmon nigiri was bad. Not worth what they're asking.	1
Excellent pizza! Slices are fantastic, prices are reasonable.	?

make use of
masked LM

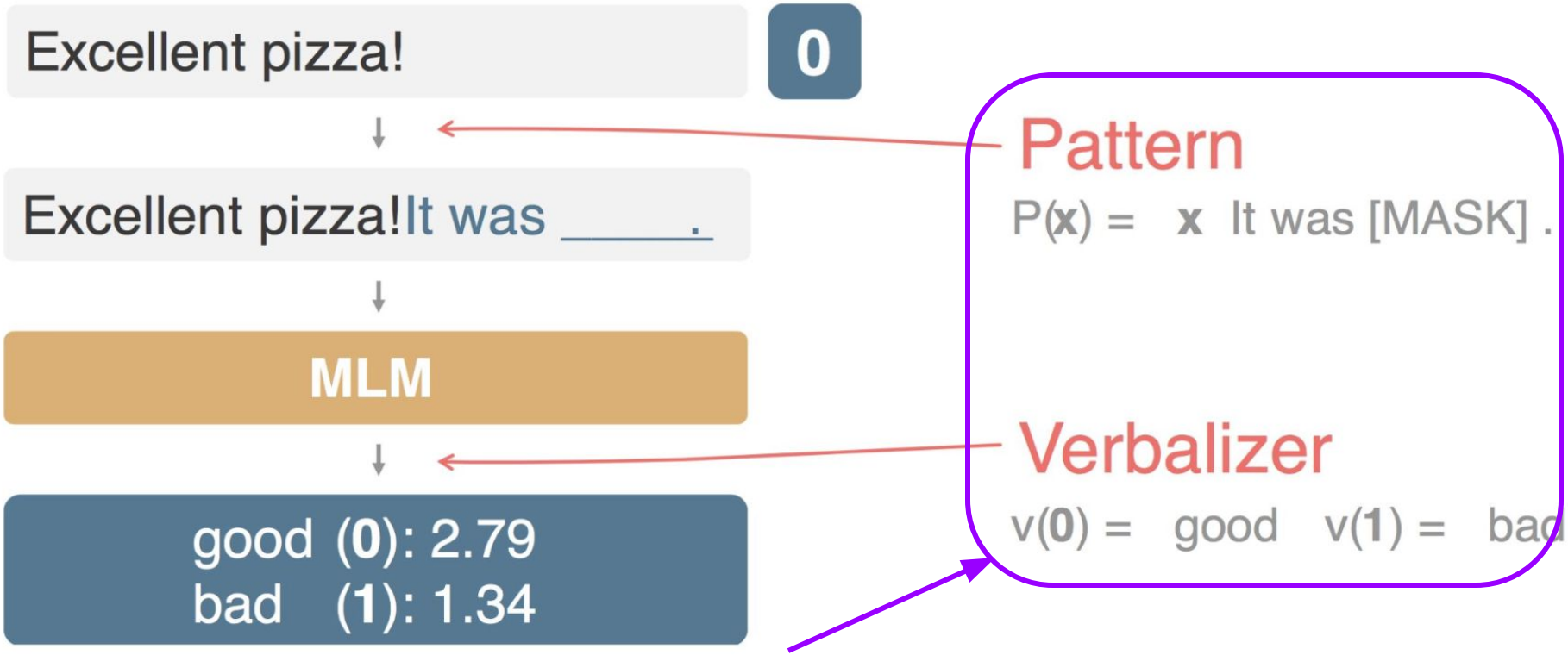
Excellent pizza! Slices are fantastic, prices are reasonable.
The restaurant is _____ .

0 = good
1 = bad

Pattern-Exploiting Training (PET) (Schick & Schütze, 2020/2021)



Pattern-Exploiting Training (PET) (Schick & Schütze, 2020/2021)



Prompting: carefully designed instruction for inquiring LLMs' knowledge (i.e., LLM-oriented)

What are "instructions"?

LLM-oriented instructions (e.g., prompts)

Human-oriented instructions (e.g., Amazon Mturk guidelines)

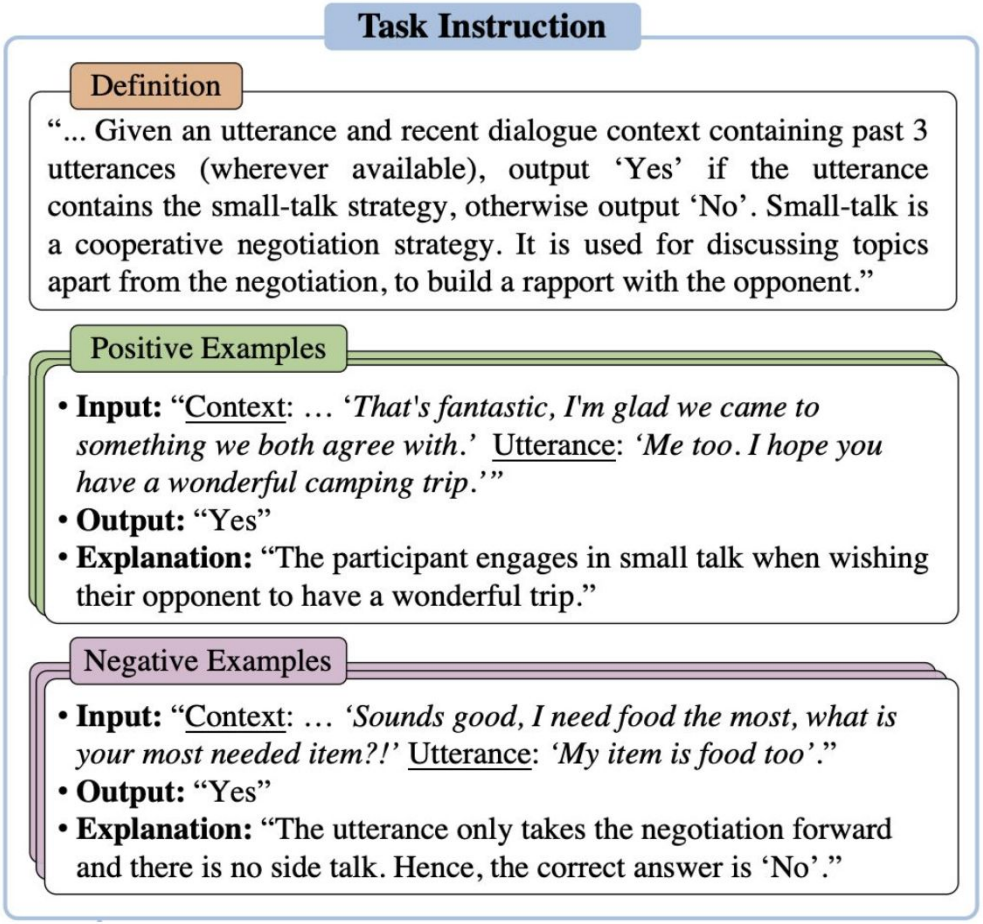
Human-oriented instructions

Natural-Instructions (Mishra et al., 2022;Wang et al., 2022)

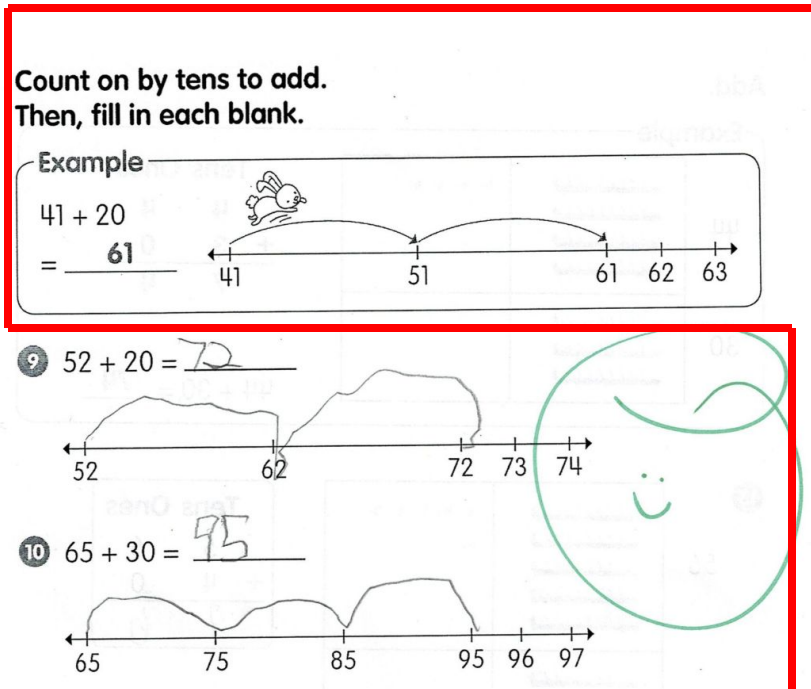
This type of instructions pretends the machine is human, and

- describes the task semantics, or
- describes how to reach the output

from the input



Human-oriented instructions



A couple of demonstrations are optionally added as a part of the instruction.

Task Instruction

Definition

"... Given an utterance and recent dialogue context containing past 3 utterances (wherever available), output 'Yes' if the utterance contains the small-talk strategy, otherwise output 'No'. Small-talk is a cooperative negotiation strategy. It is used for discussing topics apart from the negotiation, to build a rapport with the opponent."

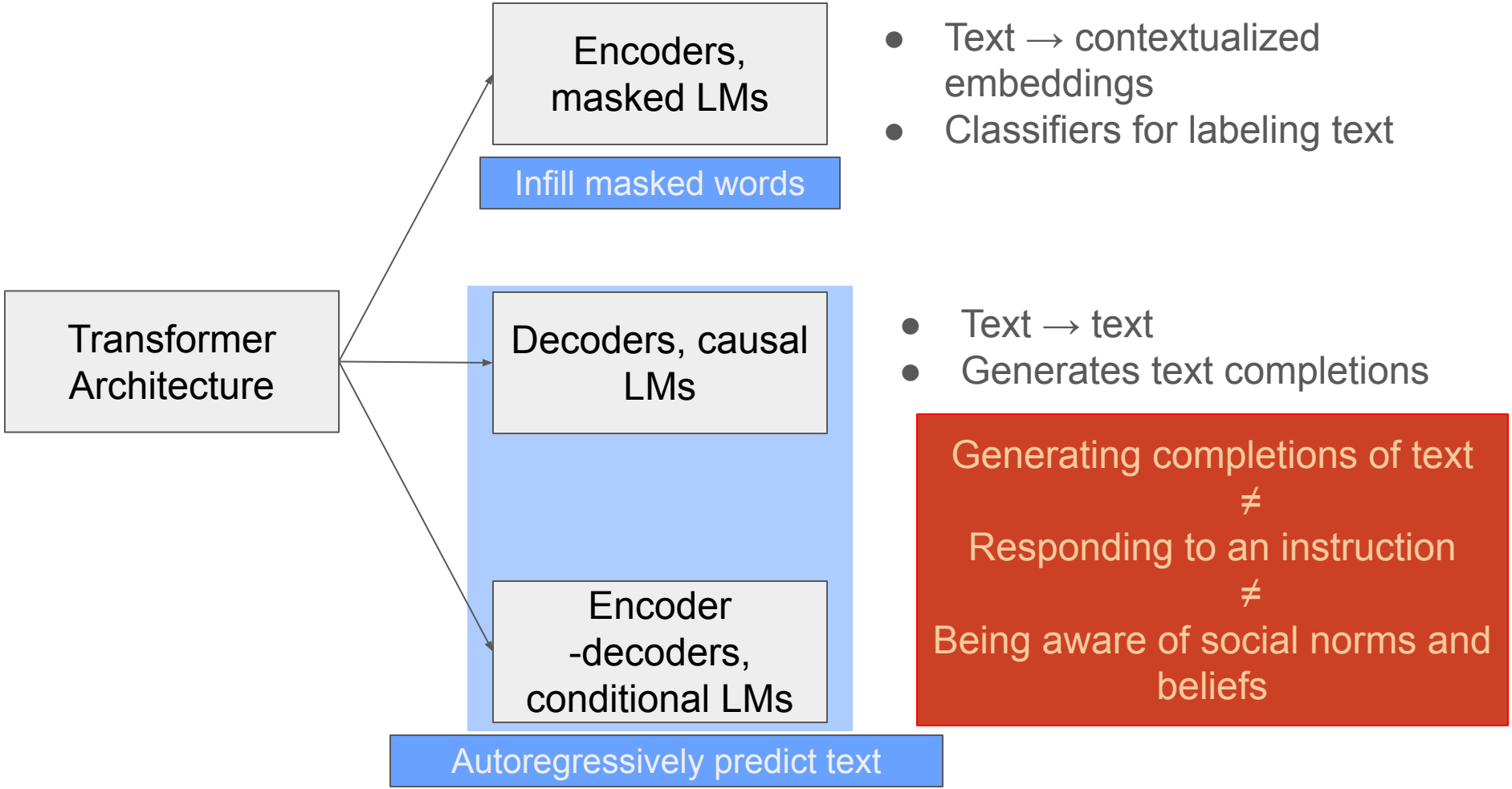
Positive Examples

- **Input:** "Context: … *'That's fantastic, I'm glad we came to something we both agree with.'* Utterance: *'Me too. I hope you have a wonderful camping trip.'*"
- **Output:** "Yes"
- **Explanation:** "The participant engages in small talk when wishing their opponent to have a wonderful trip."

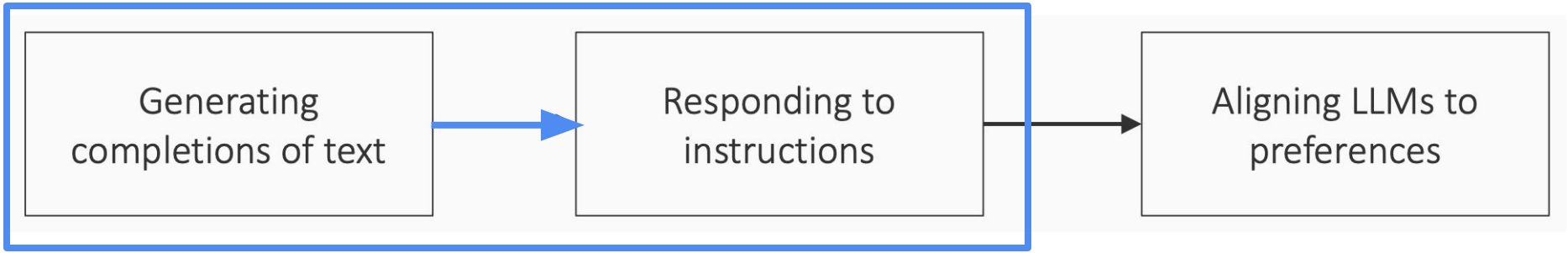
Negative Examples

- **Input:** "Context: … *'Sounds good, I need food the most, what is your most needed item?!'* Utterance: *'My item is food too'*."
- **Output:** "Yes"
- **Explanation:** "The utterance only takes the negotiation forward and there is no side talk. Hence, the correct answer is 'No'."

Instructions


Transformer Architecture
Encoders, masked LMs
Infill masked words
Decoders, causal LMs
Encoder -decoders, conditional LMs
Autoregressively predict text
Text → contextualized embeddings
Classifiers for labeling text
Text → text
Generates text completions
Generating completions of text
≠
Responding to an instruction
≠
Being aware of social norms and beliefs

This lecture



We will focus on the other arrow in the next lecture

Language modeling ≠ following instructions

What is a reasonable response to the following input?

Serendipity means the occurrence and development of events by chance in a happy or beneficial way. Use the word in a sentence.

Maybe this:

Running into Margaret and being introduced to Tom was a fortunate stroke of serendipity.

GPT 3 175B response

Serendipity is the ability to see something good in something bad. Use the word in a sentence.

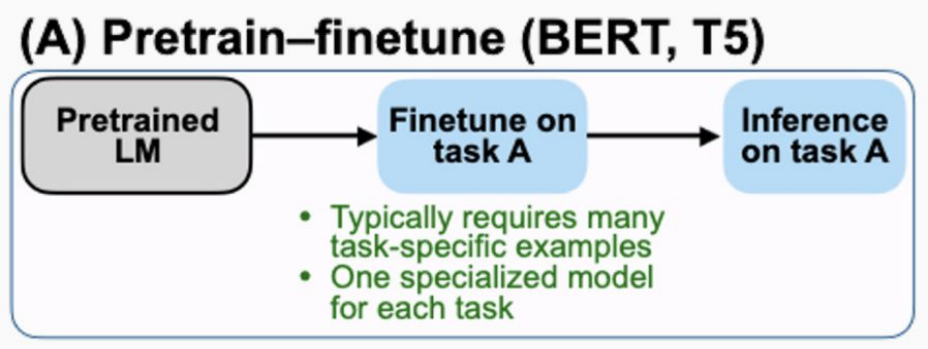
Serendipity means to be happy with what you have. Use the word in a sentence.

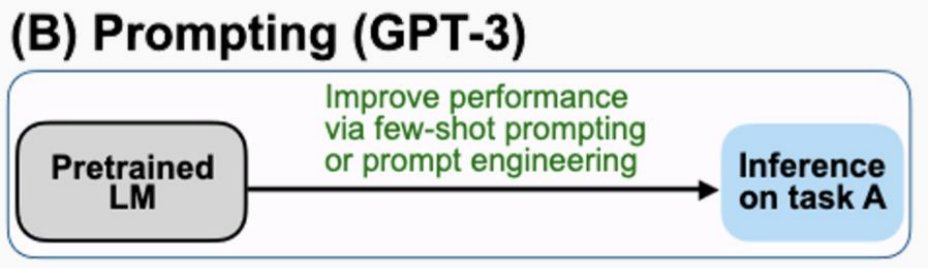
Serendip
sentence

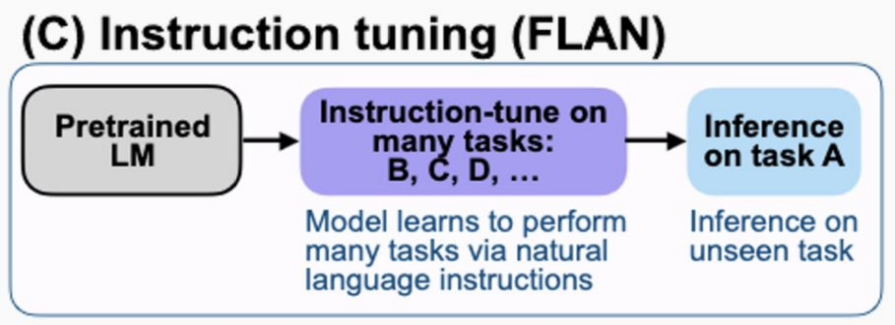
Why does the language model predict such an output?
Can you explain this based on what we know about its training?

Ouyang et al. *Training language models to follow instructions with human feedback*. NeurIPS 2022.

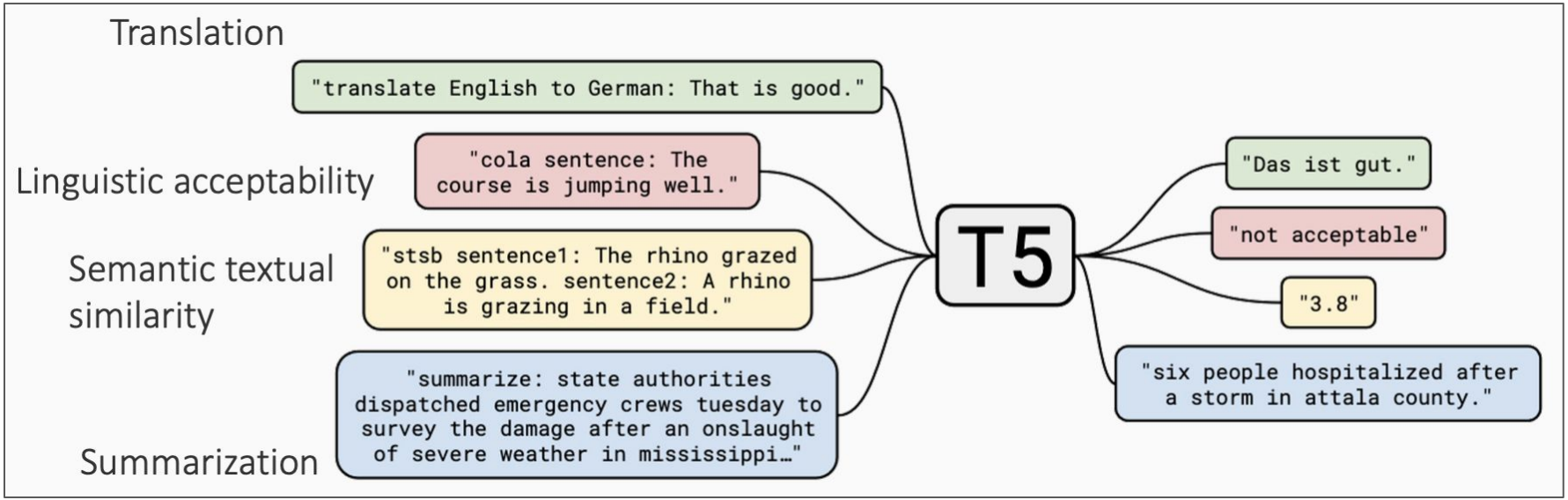
Instruction tuning







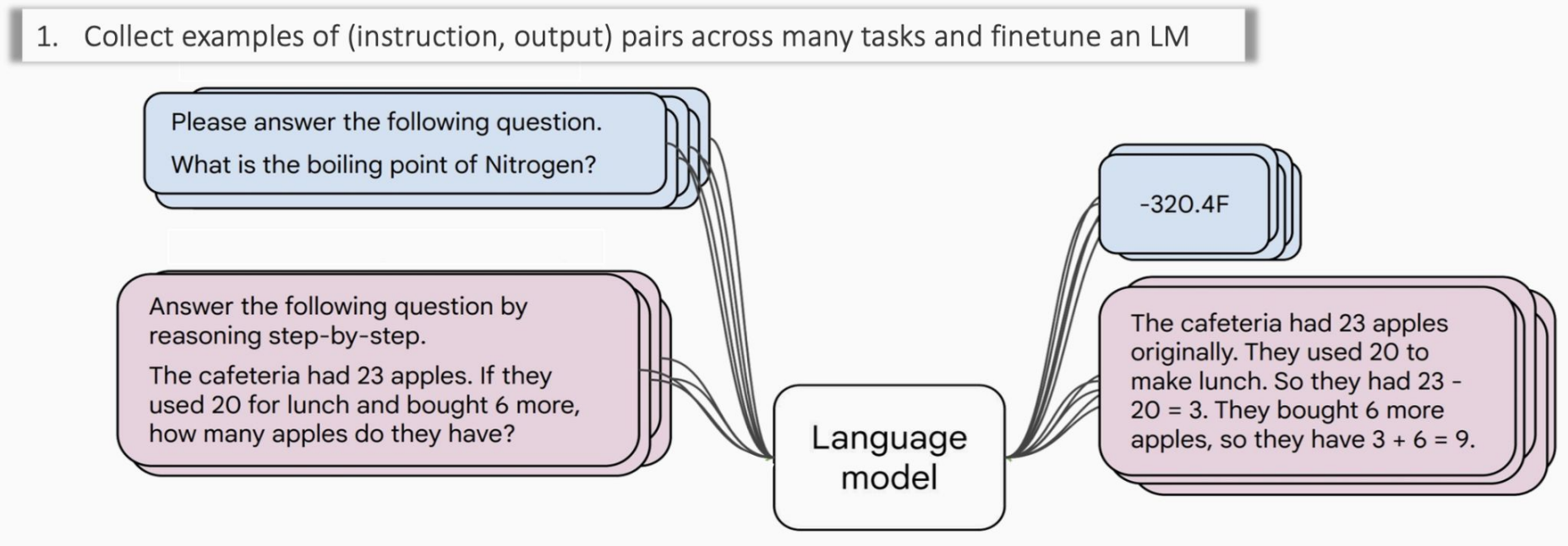
T5: “All text processing tasks → text-to-text format”



For each task, design a template so that the input and outputs are text

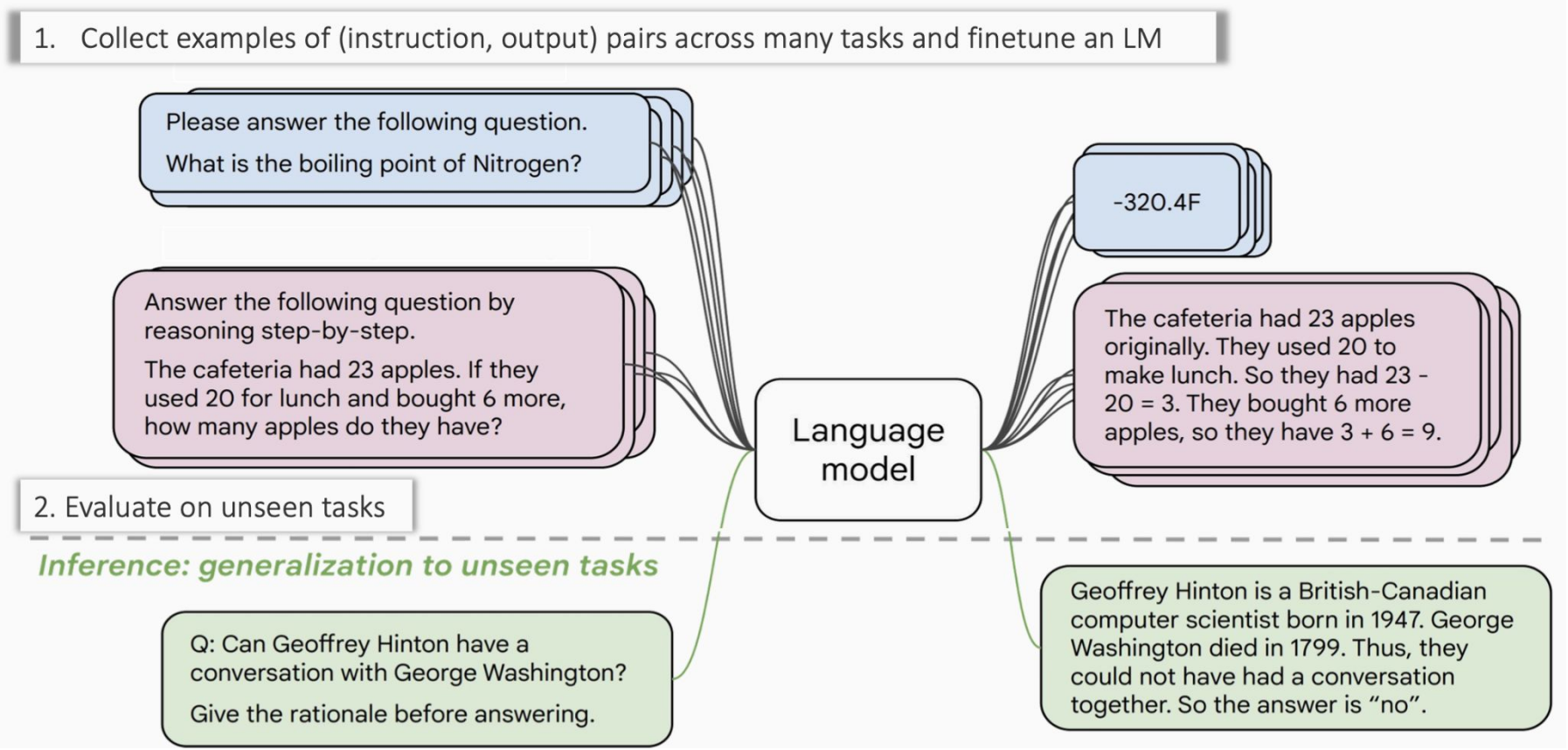
(Some previous papers had also explored this idea)

Instructions fine-tuning



Inputs and outputs are both text. The output is not a completion of the input text (as with the language modeling objective), but the response to it.

Instructions finetunning



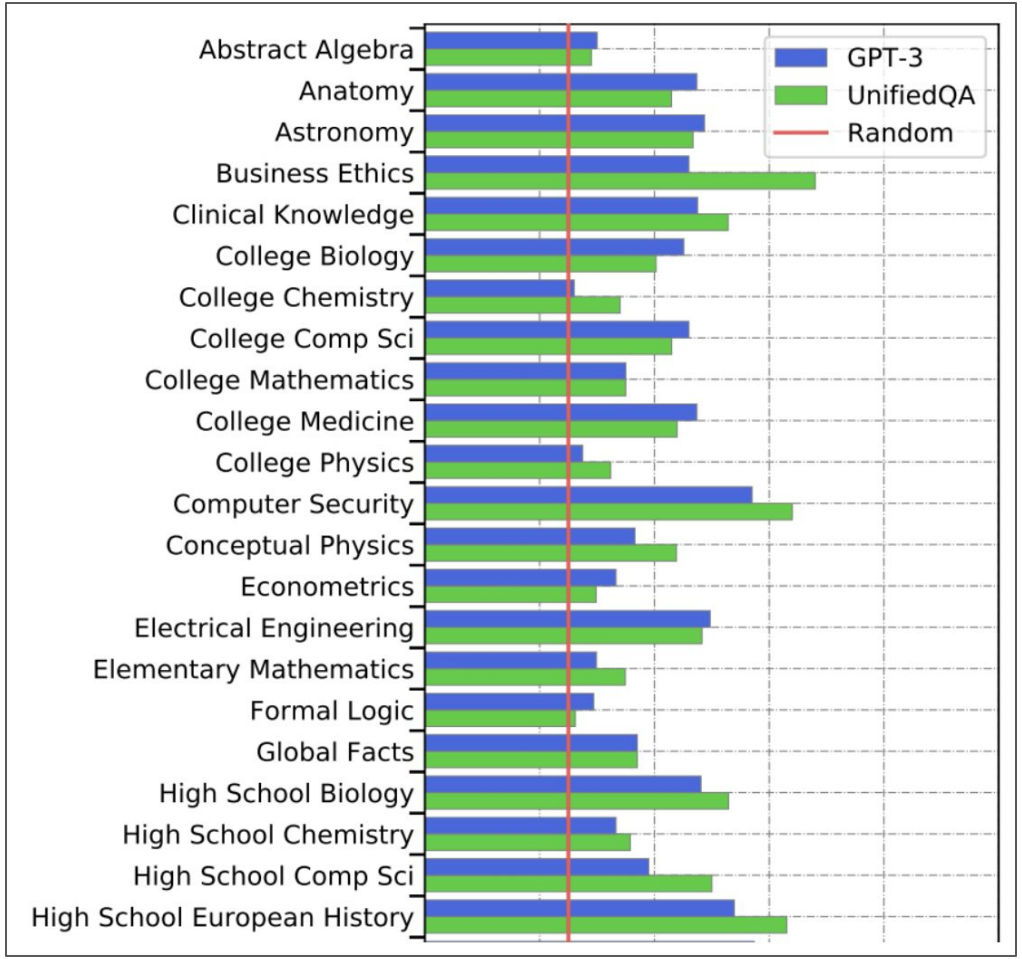
There are many instruction-tuning datasets out there

Release	Collection	Model	Model Details: Base	Model Details: Size	Model Details: Public?	Data Collection & Training Details: Prompt Types	Data Collection & Training Details: Tasks in Flan	Data Collection & Training Details: # Exs	Data Collection & Training Details: Methods
2020 05	UnifiedQA	UnifiedQA	RoBerta	110-340M	P	ZS	46 / 46	750k	
2021 04	CrossFit	BART-CrossFit	BART	140M	NP	FS	115 / 159	71.M	
2021 04	Natural Inst v1.0	Gen. BART	BART	140M	NP	ZS / FS	61 / 61	620k	+ Detailed k-shot Prompts
2021 09	Flan 2021	Flan-LaMDA	LaMDA	137B	NP	ZS / FS	62 / 62	4.4M	+ Template Variety
2021 10	P3	T0, T0+, T0++	T5-LM	3-11B	P	ZS	62 / 62	12M	+ Template Variety + Input Inversion
2021 10	MetaICL	MetaICL	GPT-2	770M	P	FS	100 / 142	3.5M	+ Input Inversion + Noisy Channel Opt
2021 11	ExMix	ExT5	T5	220M-11B	NP	ZS	72 / 107	500k	+ With Pretraining
2022 04	Super-Natural Inst.	Tk-Instruct	T5-LM, mT5	11-13B	P	ZS / FS	1556 / 1613	5M	+ Detailed k-shot Prompts + Multilingual
2022 10	GLM	GLM-130B	GLM	130B	P	FS	65 / 77	12M	+ With Pretraining + Bilingual (en, zh-cn)
2022 11	xP3	BLOOMz, mT0	BLOOM, mT5	13-176B	P	ZS	53 / 71	81M	+ Massively Multilingual
2022 12	Unnatural Inst.†	T5-LM-Unnat. Inst.	T5-LM	11B	NP	ZS	~20 / 117	64k	+ Synthetic Data
2022 12	Self-Instruct†	GPT-3 Self Inst.	GPT-3	175B	NP	ZS	Unknown	82k	+ Synthetic Data + Knowledge Distillation
2022 12	OPT-IML Bench†	OPT-IML	OPT	30-175B	P	ZS + FS, CoT	~2067 / 2207	18M	+ Template Variety + Input Inversion + Multilingual
2022 10	Flan 2022 (ours)	Flan-T5, Flan-PaLM	T5-LM, PaLM	10M-540B	P NP	ZS + FS, CoT	1836	15M	+ Template Variety + Input Inversion + Multilingual

Benchmarks for multitask LMs

Massive Multitask Language Understanding (MMLU)

[Hendrycks et al., 2021]



Some intuition: examples from MMLU

Astronomy

What is true for a type-Ia supernova?

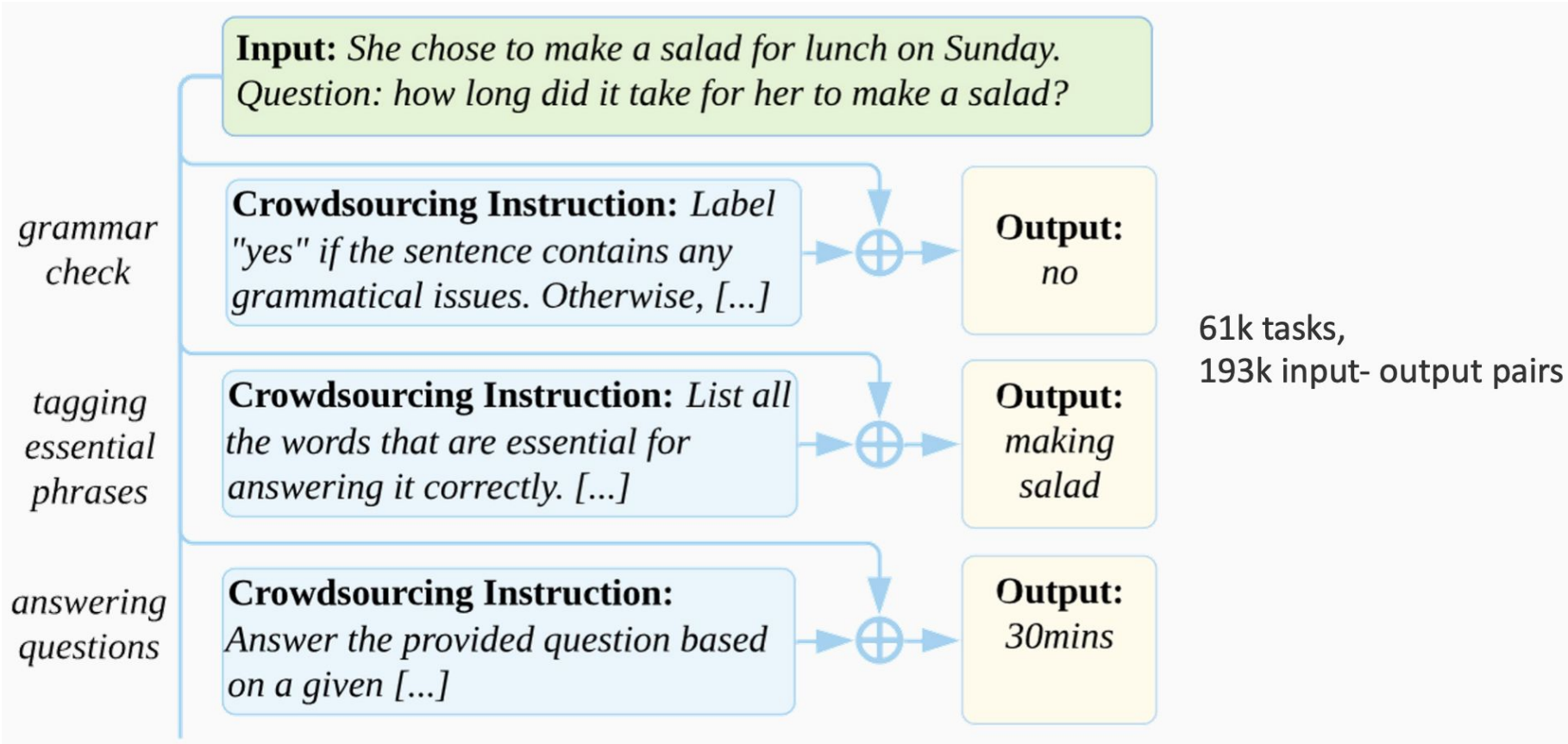
A. This type occurs in binary systems.
B. This type occurs in young galaxies.
C. This type produces gamma-ray bursts.
D. This type produces high amounts of X-rays.

High School Biology

In a population of giraffes, an environmental change occurs that favors individuals that are tallest. As a result, more of the taller individuals are able to obtain nutrients and survive to pass along their genetic information. This is an example of

A. directional selection.
B. stabilizing selection.
C. sexual selection.
D. disruptive selection

Natural instructions



Mishra, Swaroop, et al. "*Cross-Task Generalization via Natural Language Crowdsourcing Instructions.*" ACL'22.

Super-Natural Instructions

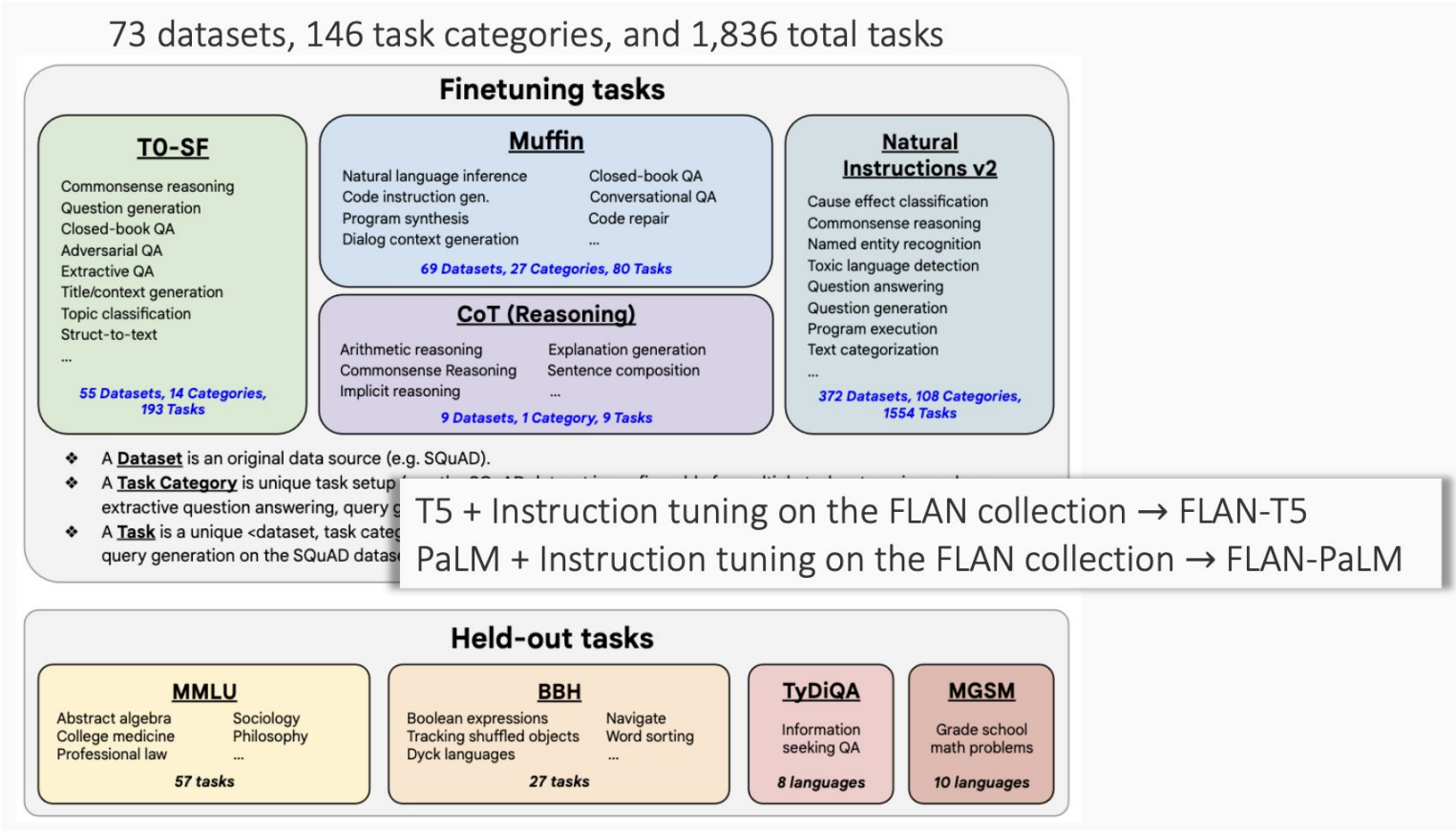
Super-NaturalInstructions dataset contains over 1.6K tasks, 3M+ examples

Classification, sequence tagging, rewriting, translation, QA...

Many languages: 576 non-English

Wang 2022. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks

The FLAN collection



Chung, Hyung Won, et al. "*Scaling instruction-finetuned language models.*" arXiv preprint arXiv:2210.11416 (2022).

Instruction-Tuning: Example

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:
(A) They will discuss the reporter's favorite dishes
(B) They will discuss the chef's favorite dishes
(C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.
The reporter and the chef will discuss the reporter's favorite dishes.
The reporter and the chef will discuss the chef's favorite dishes.
The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✖ (doesn't answer question)

https://huggingface.co/google/flan-t5-xxl

Chung, Hyung Won, et al. "Scaling instruction-finetuned language models." arXiv preprint arXiv:2210.11416 (2022).

Instruction-Tuning: Example

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:
(A) They will discuss the reporter's favorite dishes
(B) They will discuss the chef's favorite dishes
(C) Ambiguous

A: Let's think step by step.

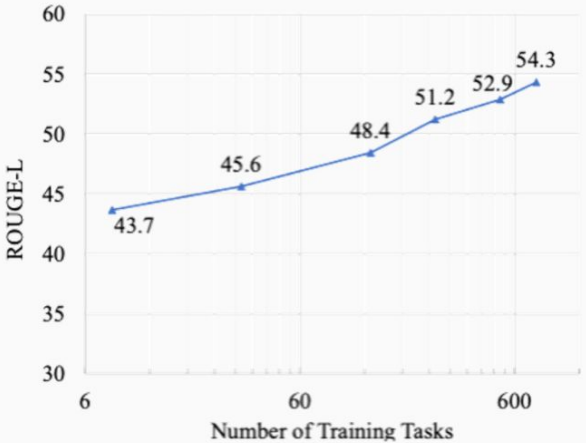
After instruction finetuning

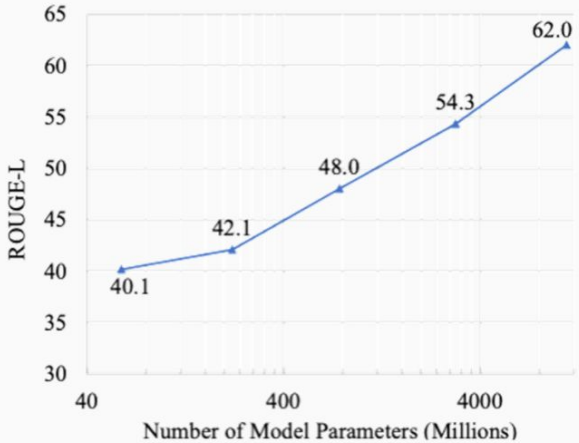
The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C). ✅

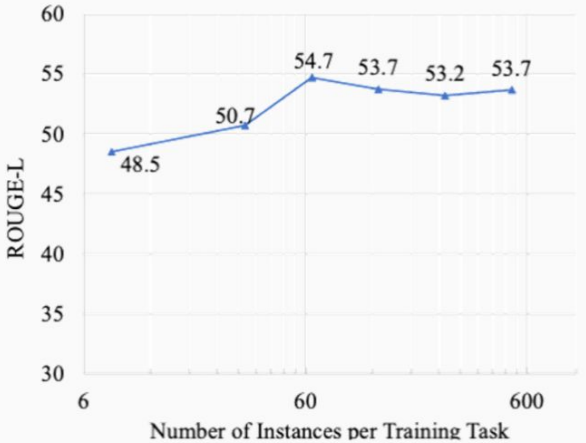
https://huggingface.co/google/flan-t5-xxl

Chung, Hyung Won, et al. "Scaling instruction-finetuned language models." arXiv preprint arXiv:2210.11416 (2022).

Scaling Instruction-Tuning







Linear growth of model performance with exponential increase in observed tasks and model size.

Number of examples has little effect.

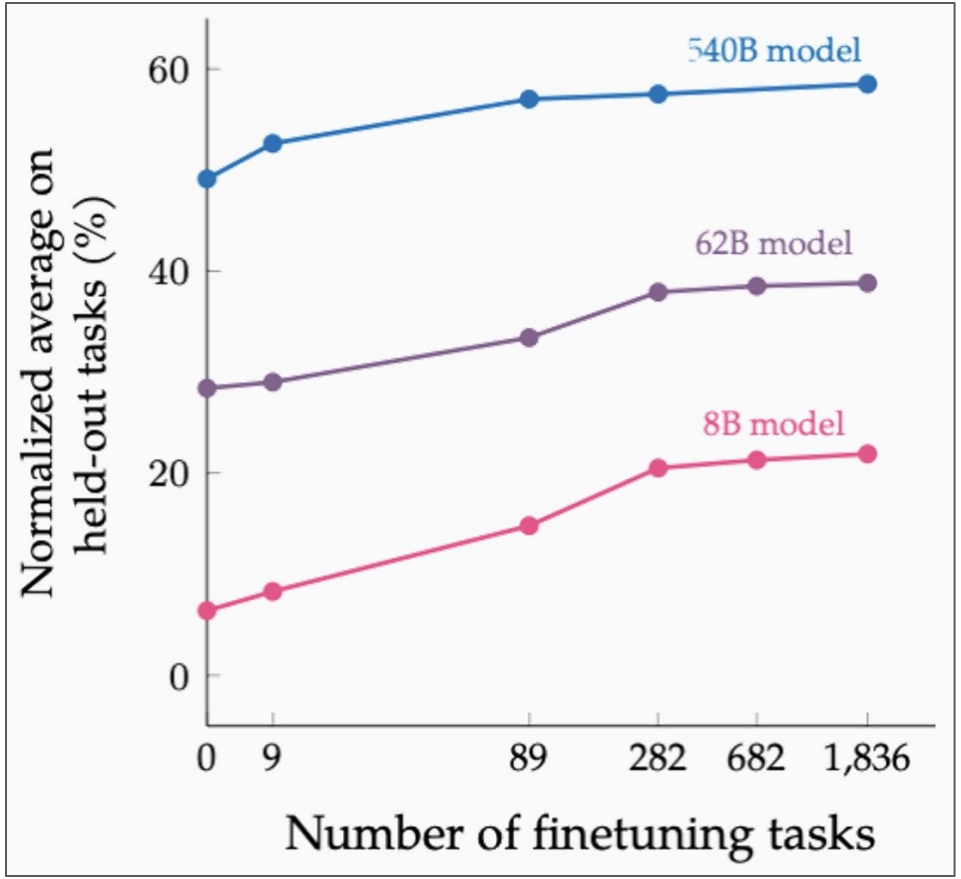
Wang 2022. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks

Scaling Instruction-Tuning

Instruction finetuning improves performance by a large margin **compared to no finetuning**

Increasing the **number** of finetuning **tasks** improves **performance**

Increasing model **scale** by an order of magnitude (i.e., 8B → 62B or 62B → 540B) **improves performance** substantially for both finetuned and non-finetuned models

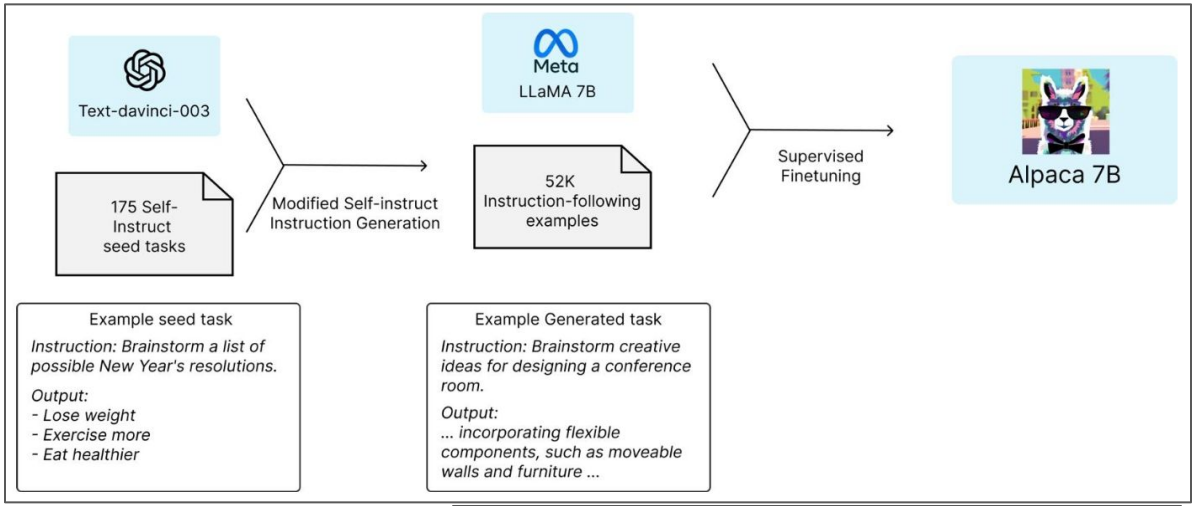


Chung, Hyung Won, et al. "Scaling instruction-finetuned language models." arXiv preprint arXiv:2210.11416 (2022).

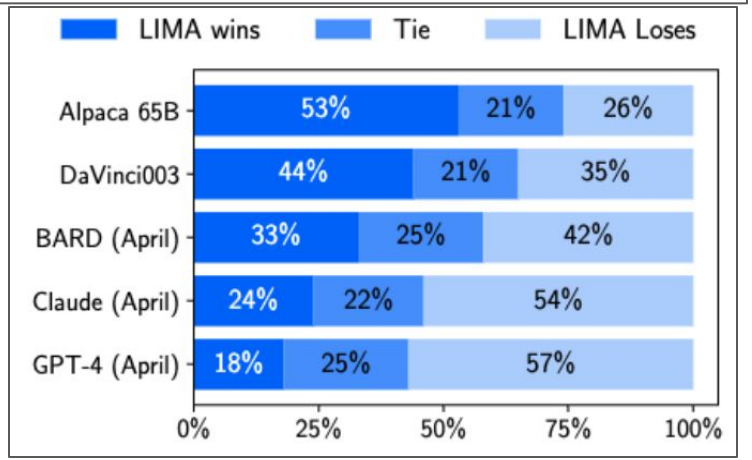
What have we learned from this?

Generate instructions, input, and output from a LM *[Wang et al., 2022]*

Alpaca: fine-tuned from the LLaMA 7B model on 52K instruction-following examples



• You don't need many samples to instruction tune (e.g., "*LIMA: Less Is More for Alignment*" Zhou et al., 2023)



Let's build from scratch

Fine-tuning a pretrained LLM

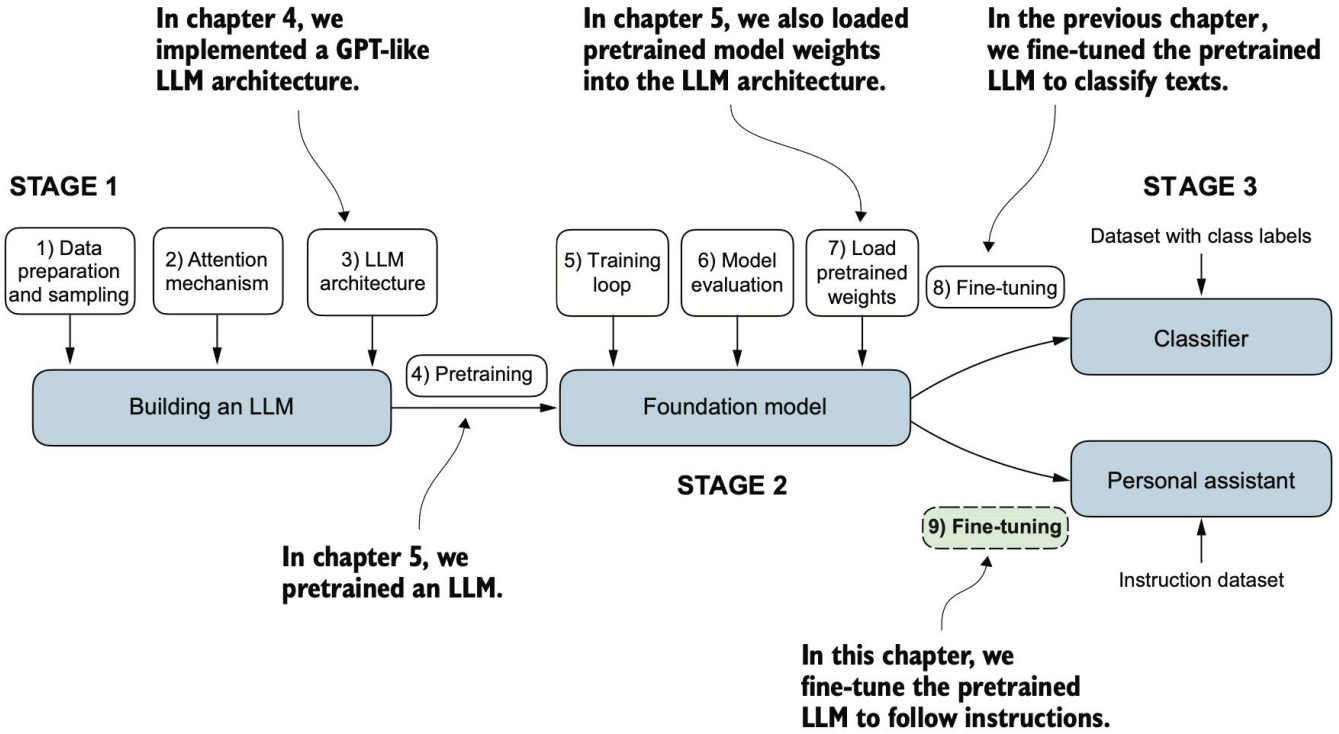


Figure 7.1 The three main stages of coding an LLM. This chapter focuses on step 9 of stage 3: fine-tuning a pretrained LLM to follow human instructions.

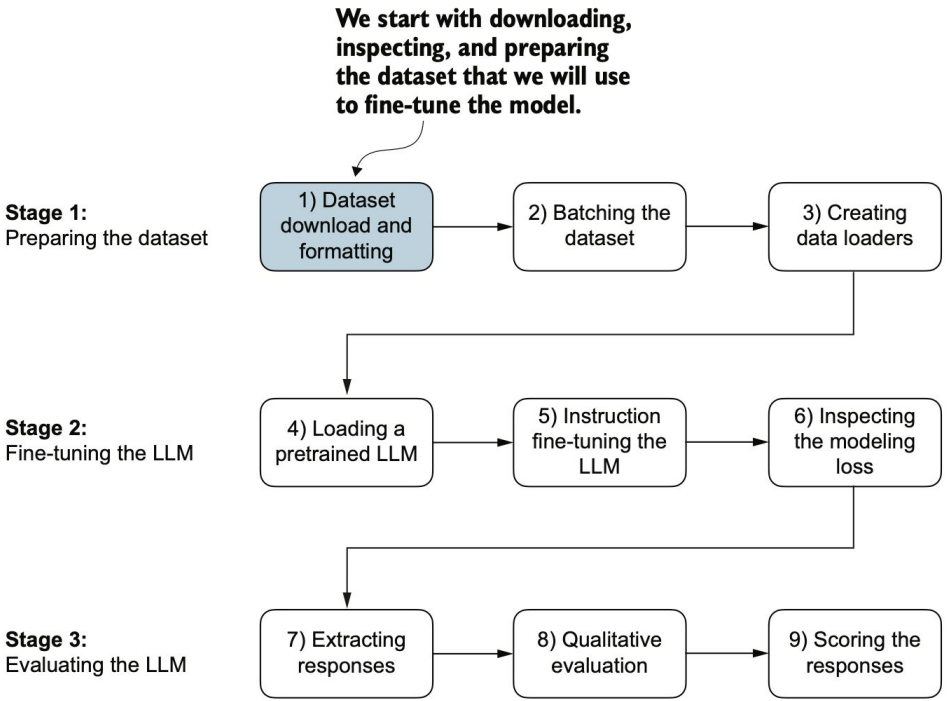


Figure 7.3 The three-stage process for instruction fine-tuning an LLM. Stage 1 involves dataset preparation, stage 2 focuses on model setup and fine-tuning, and stage 3 covers the evaluation of the model. We will begin with step 1 of stage 1: downloading and formatting the dataset.

Instructions

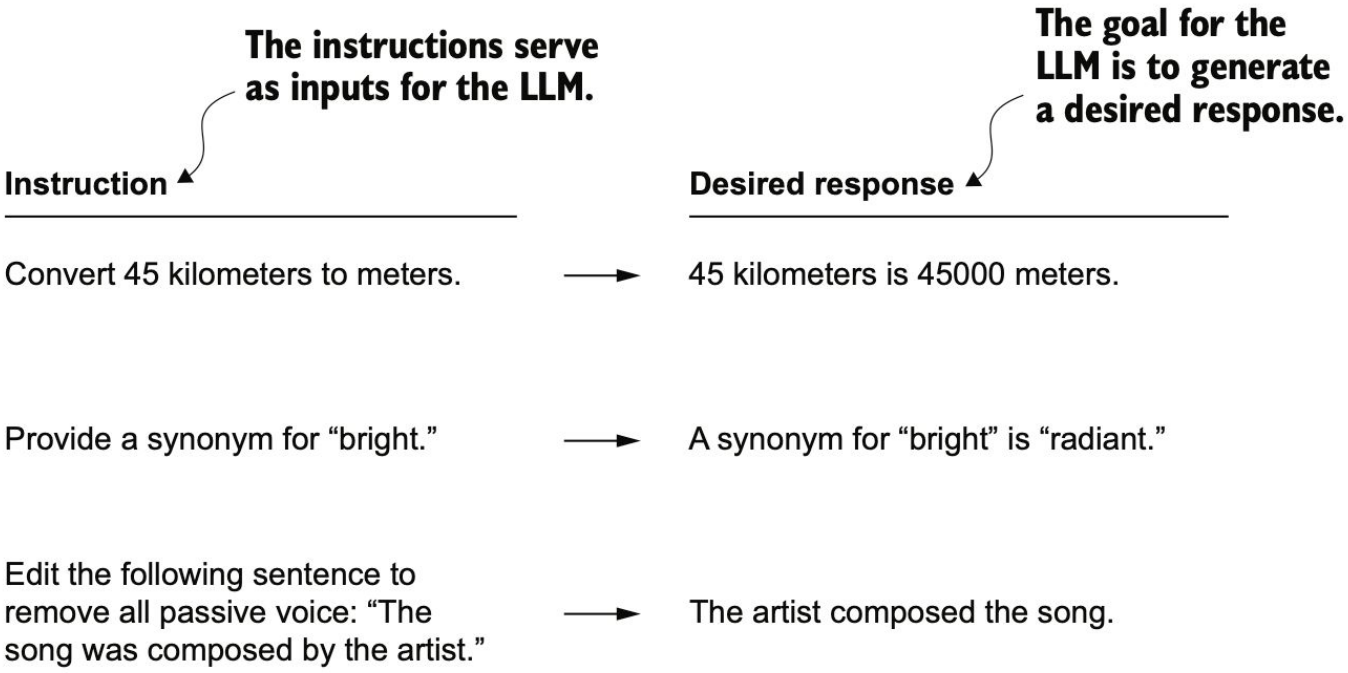


Figure 7.2 Examples of instructions that are processed by an LLM to generate desired responses

Preparing the instruction dataset

1,100 *instruction-response* pairs.

Specifically created for the book.

Listing 7.1 Downloading the dataset

```
import json
import os
import urllib

def download_and_load_file(file_path, url):
    if not os.path.exists(file_path):
        with urllib.request.urlopen(url) as response:
            text_data = response.read().decode("utf-8")
        with open(file_path, "w", encoding="utf-8") as file:
            file.write(text_data)
```

The content of the example entry is

```
Example entry:
 {'instruction': 'Identify the correct spelling of the following word.',
  'input': 'Ocassion', 'output': "The correct spelling is 'Occasion.'"}
```

```
    "/main/ch07/01_main-chapter-code/instruction-data.json"
)


data = download_and_load_file(file_path, url)
print("Number of entries:", len(data))
```

The output of executing the preceding code is

```
Number of entries: 1100
```

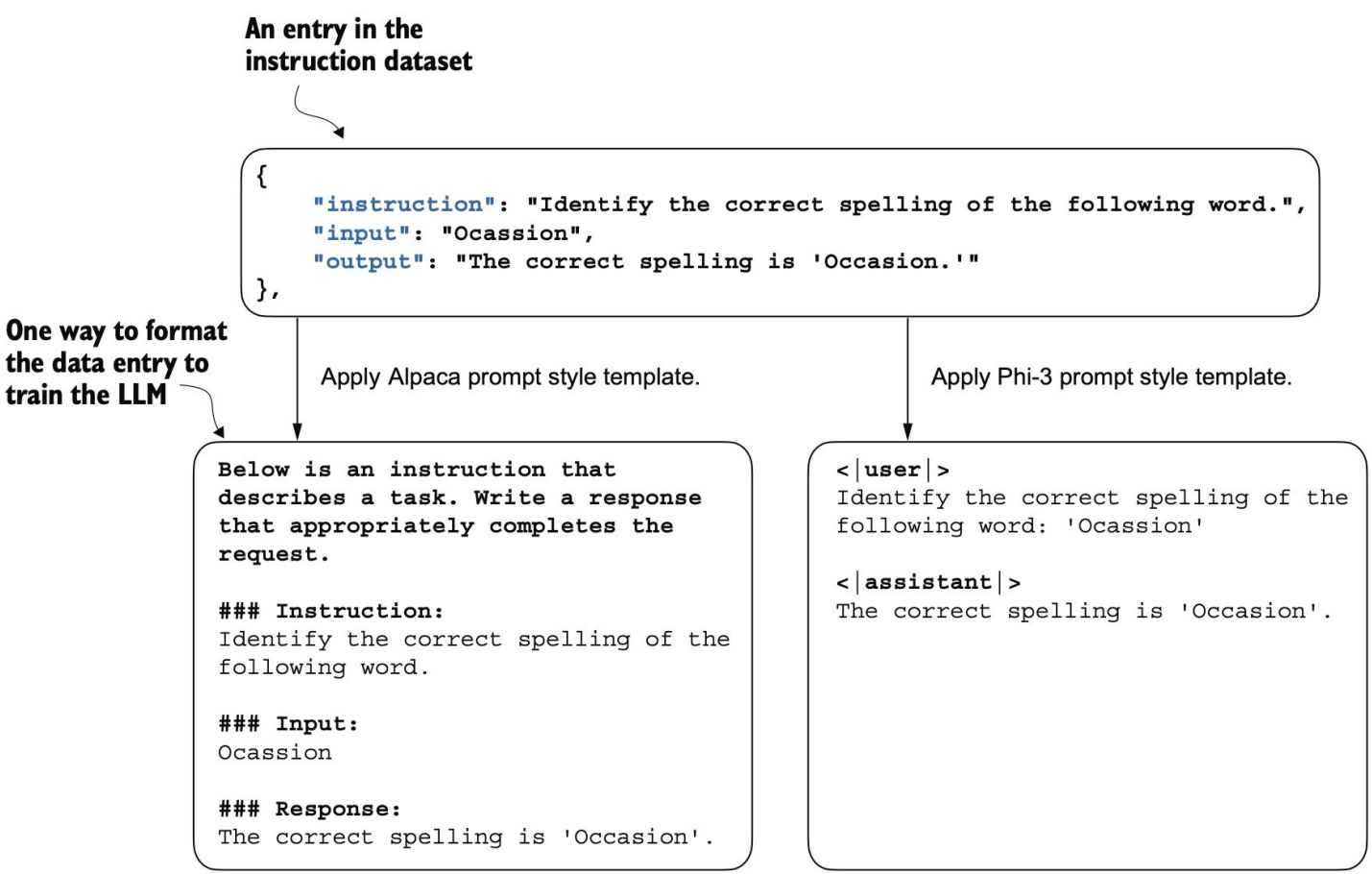


Figure 7.4 Comparison of prompt styles for instruction fine-tuning in LLMs. The Alpaca style (left) uses a structured format with defined sections for instruction, input, and response, while the Phi-3 style (right) employs a simpler format with designated <|user|> and <|assistant|> tokens.

Listing 7.2

```
def format_i
    instruct
        f"Be
        f"Wr
        f"\n
    )

    input_te
        f"\n
    )
    return instruction_text + input_text
```

```
Below is an instruction that describes a task. Write a response that
appropriately completes the request.

### Instruction:
Identify the correct spelling of the following word.

### Input:
Ocassion

### Response:
The correct spelling is 'Occasion.'
```

```
model_input = format_input(data[50])
desired_response = f"\n\n### Response:\n{data[50]['output']}"
print(model_input + desired_response)
```

Listing 7.3 Partitioning the dataset

```
train_portion = int(len(data) * 0.85)
test_portion = int(len(data) * 0.1)
val_portion = len(data) - train_portion - test_portion

train_data = data[:train_portion]
test_data = data[train_portion:train_portion + test_portion]
val_data = data[train_portion + test_portion:]

print("Training set length:", len(train_data))
print("Validation set length:", len(val_data))
print("Test set length:", len(test_data))
```

Use 85% of the data for training

Use 10% for testing

Use remaining 5% for validation

This partitioning results in the following dataset sizes:

```
Training set length: 935
Validation set length: 55
Test set length: 110
```

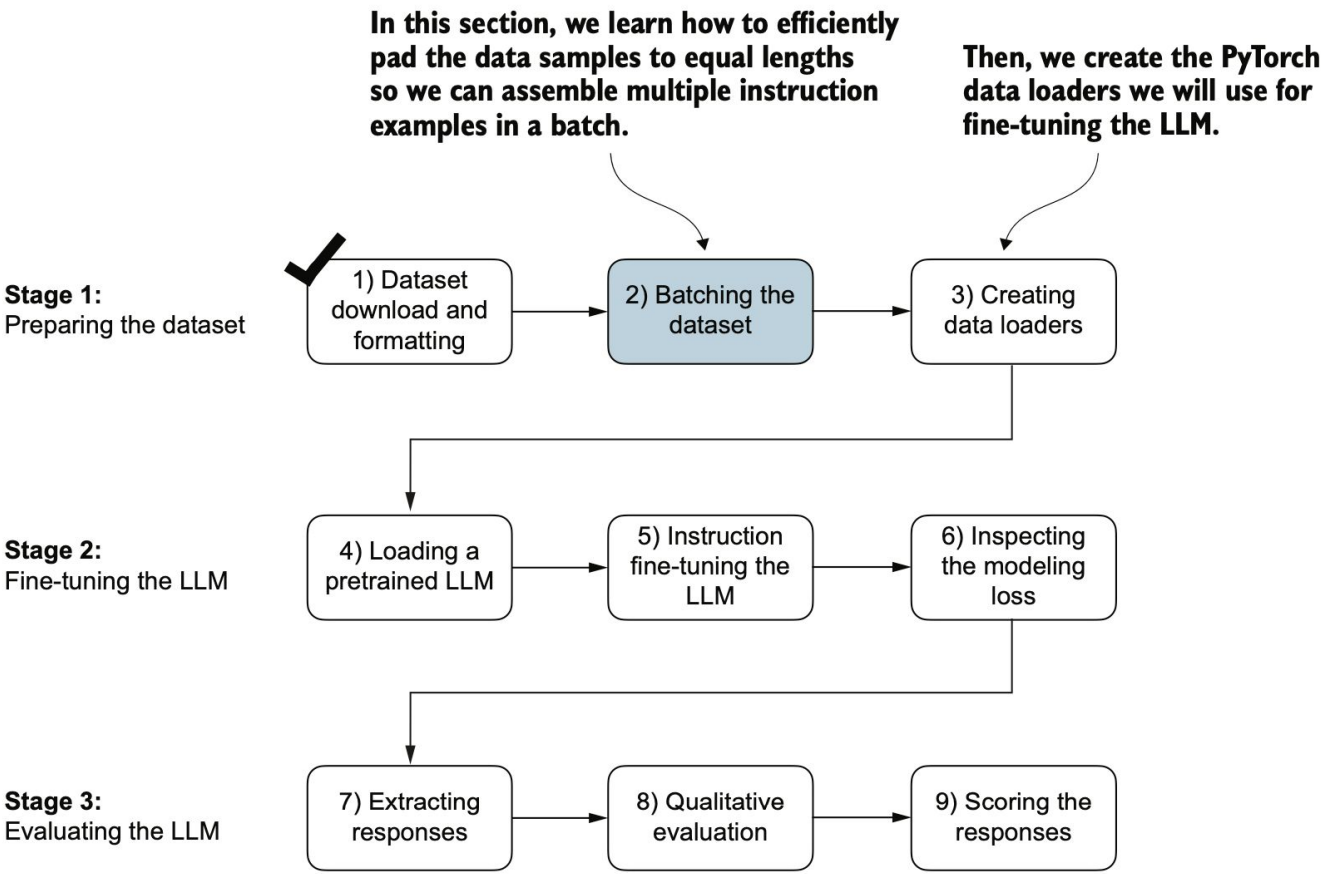


Figure 7.5 The three-stage process for instruction fine-tuning an LLM. Next, we look at step 2 of stage 1: assembling the training batches.

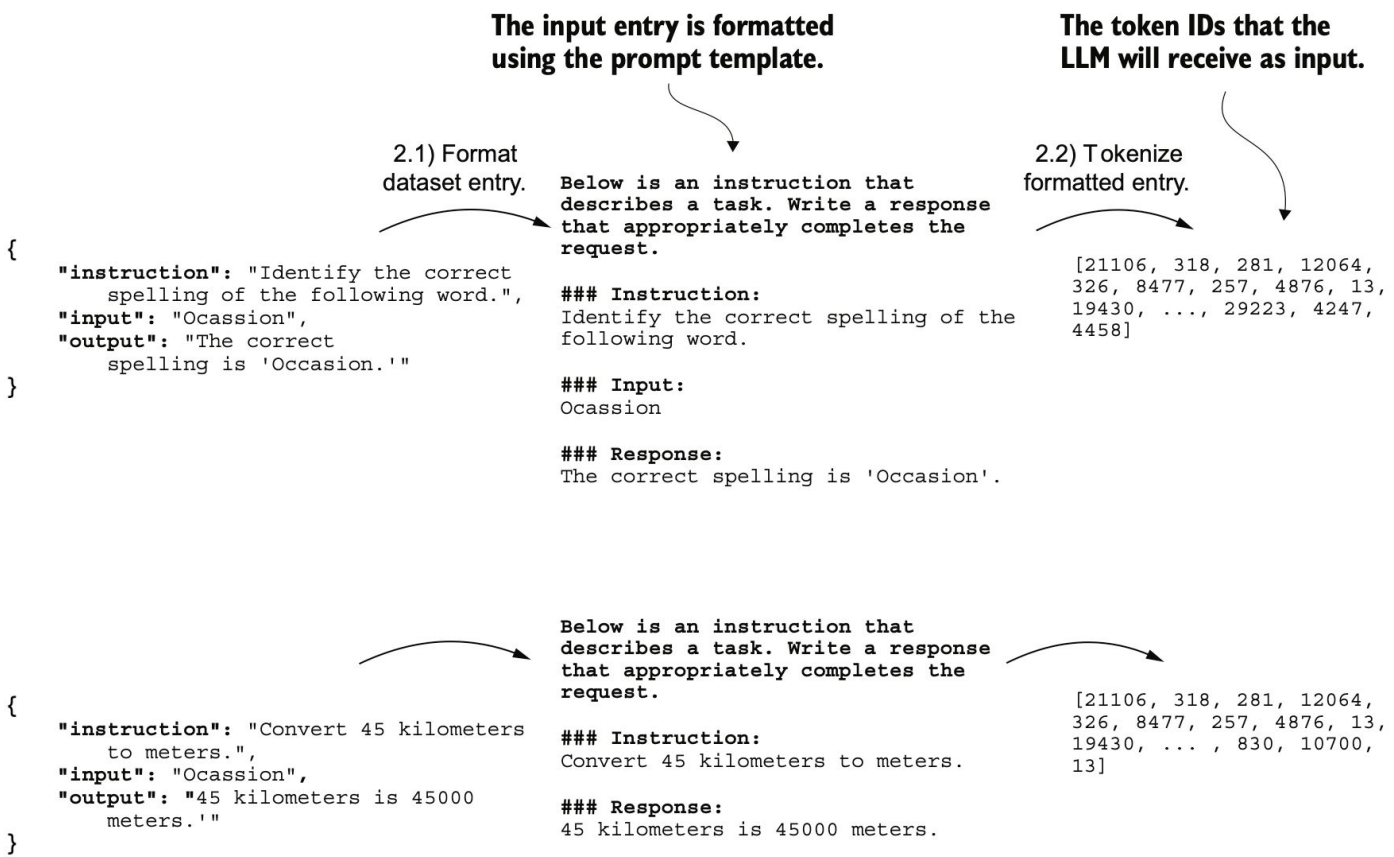


Figure 7.7 The first two steps involved in implementing the batching process. Entries are first formatted using a specific prompt template (2.1) and then tokenized (2.2), resulting in a sequence of token IDs that the model can process.

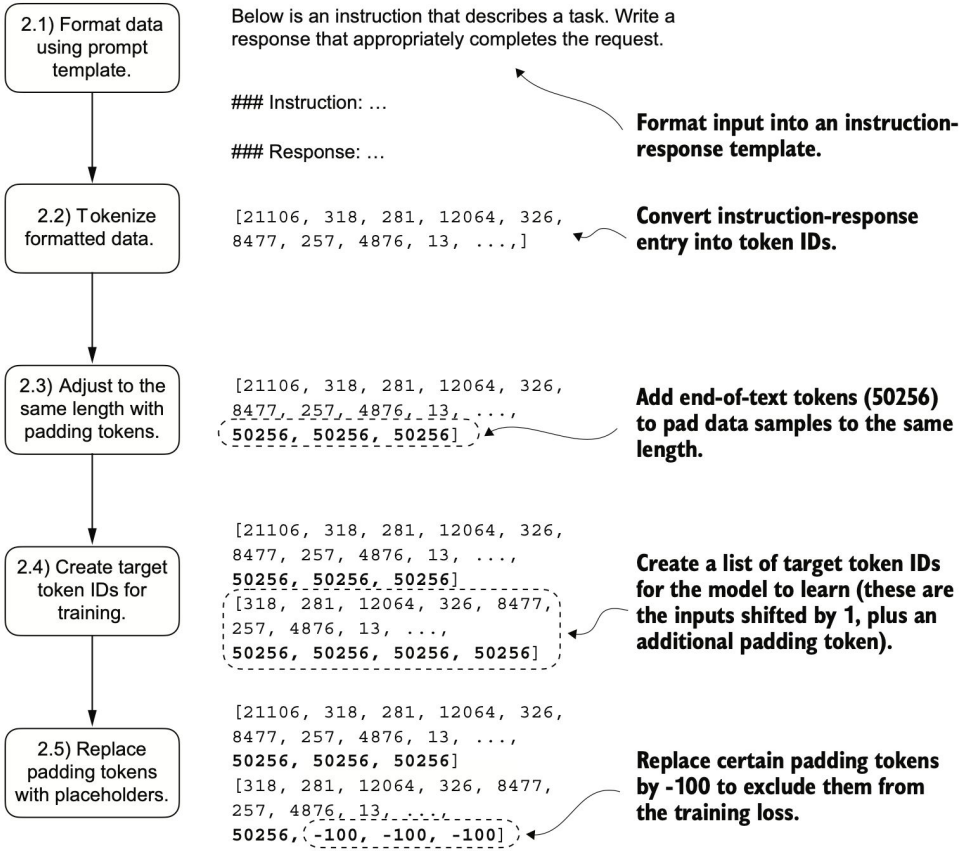


Figure 7.6 The five substeps involved in implementing the batching process: (2.1) applying the prompt template, (2.2) using tokenization from previous chapters, (2.3) adding padding tokens, (2.4) creating target token IDs, and (2.5) replacing `-100` placeholder tokens to mask padding tokens in the loss function.

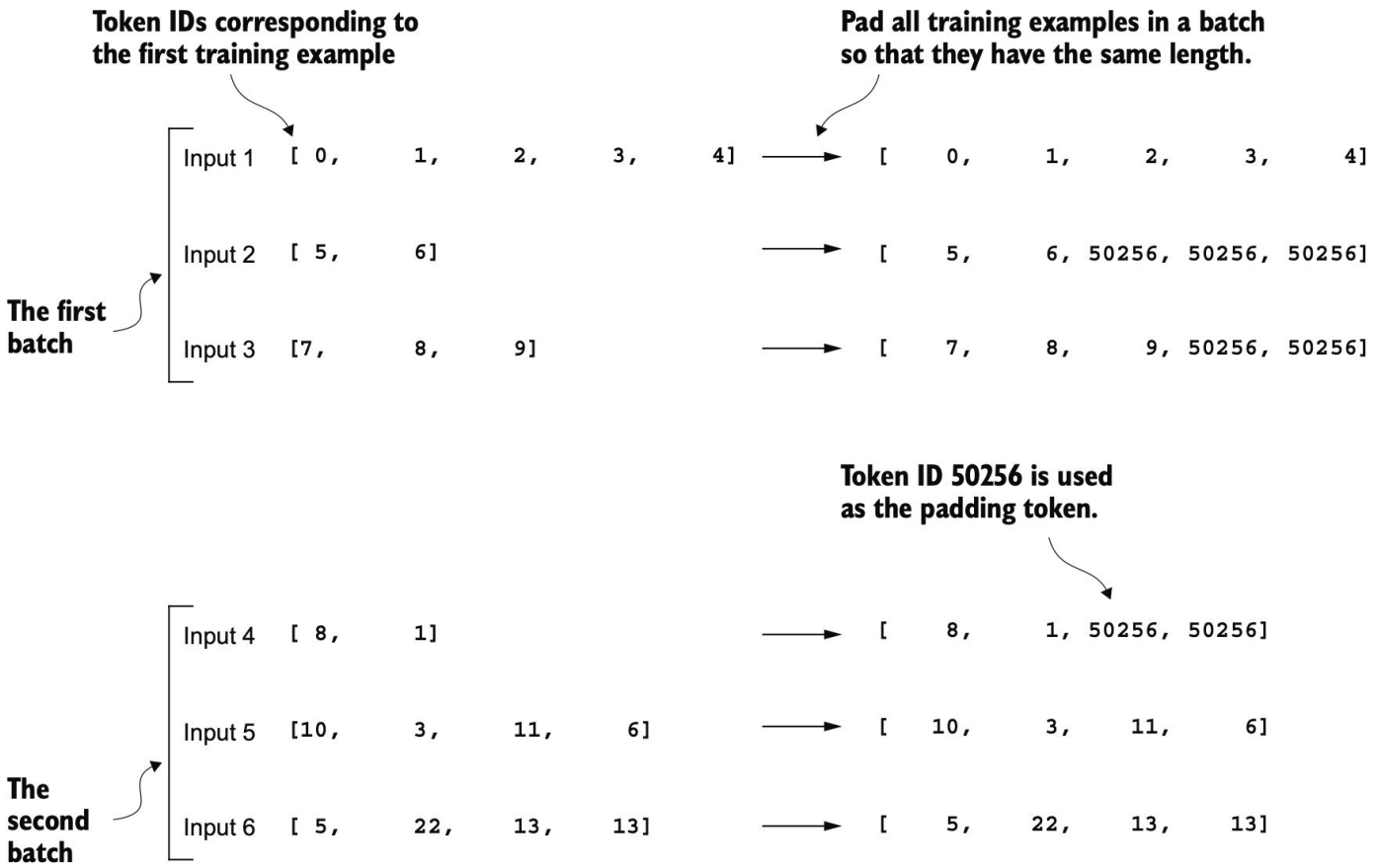


Figure 7.8 The padding of training examples in batches using token ID `50256` to ensure uniform length within each batch. Each batch may have different lengths, as shown by the first and second.

```
def custom_collate_draft_1(
    batch,
    pad_token_id=50256,
    device="cpu"
):
    batch_max_length = max(len(item)+1 for item in batch)
    inputs_lst = []

    for item in batch:
        new_item = item.copy()
        new_item += [pad_token_id]


        padded = (
            new_item + [pad_token_id] *
            (batch_max_length - len(new_item))
        )
        inputs = torch.tensor(padded[:-1])
        inputs_lst.append(inputs)

    inputs_tensor = torch.stack(inputs_lst).to(device)
    return inputs_tensor
```

Finds the longest sequence in the batch

Pads and prepares inputs

Removes extra padded token added earlier

Converts the list of inputs to a tensor and transfers it to the target device

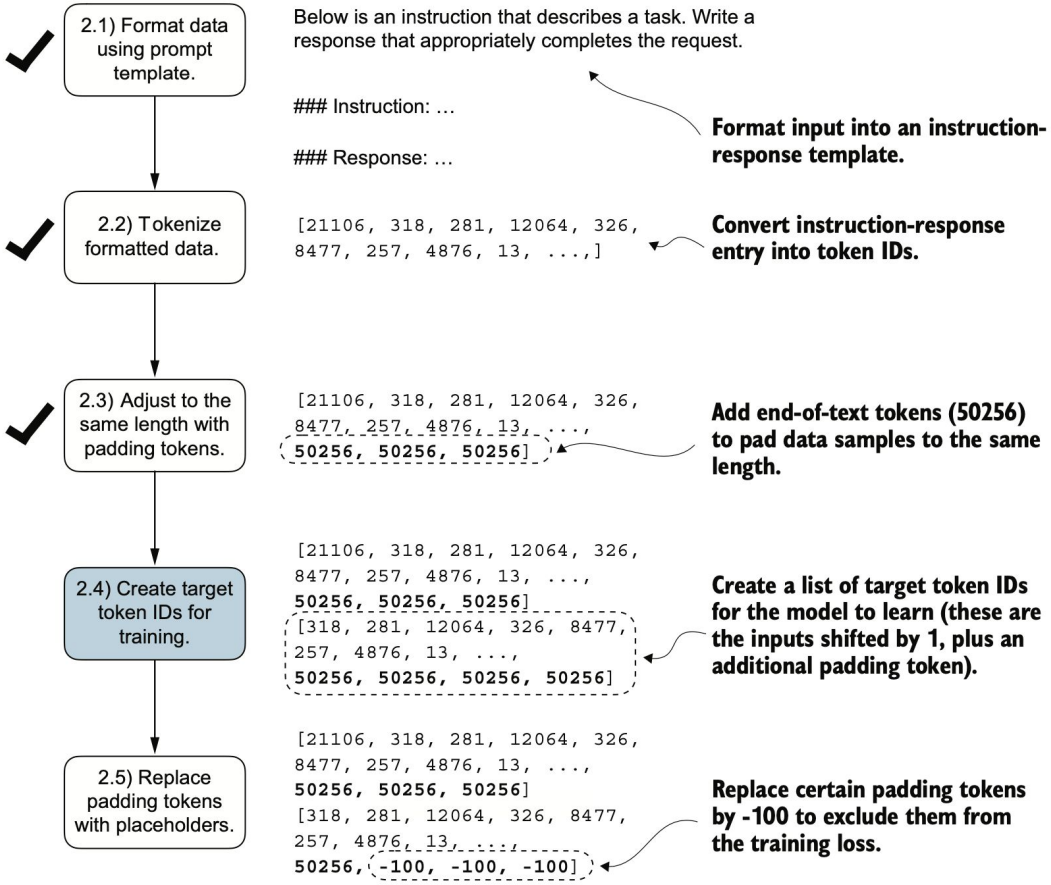


Figure 7.9 The five substeps involved in implementing the batching process. We are now focusing on step 2.4, the creation of target token IDs. This step is essential as it enables the model to learn and predict the tokens it needs to generate.

Similar to the process we used to pretrain an LLM, the target token IDs match the input token IDs but are shifted one position to the right.

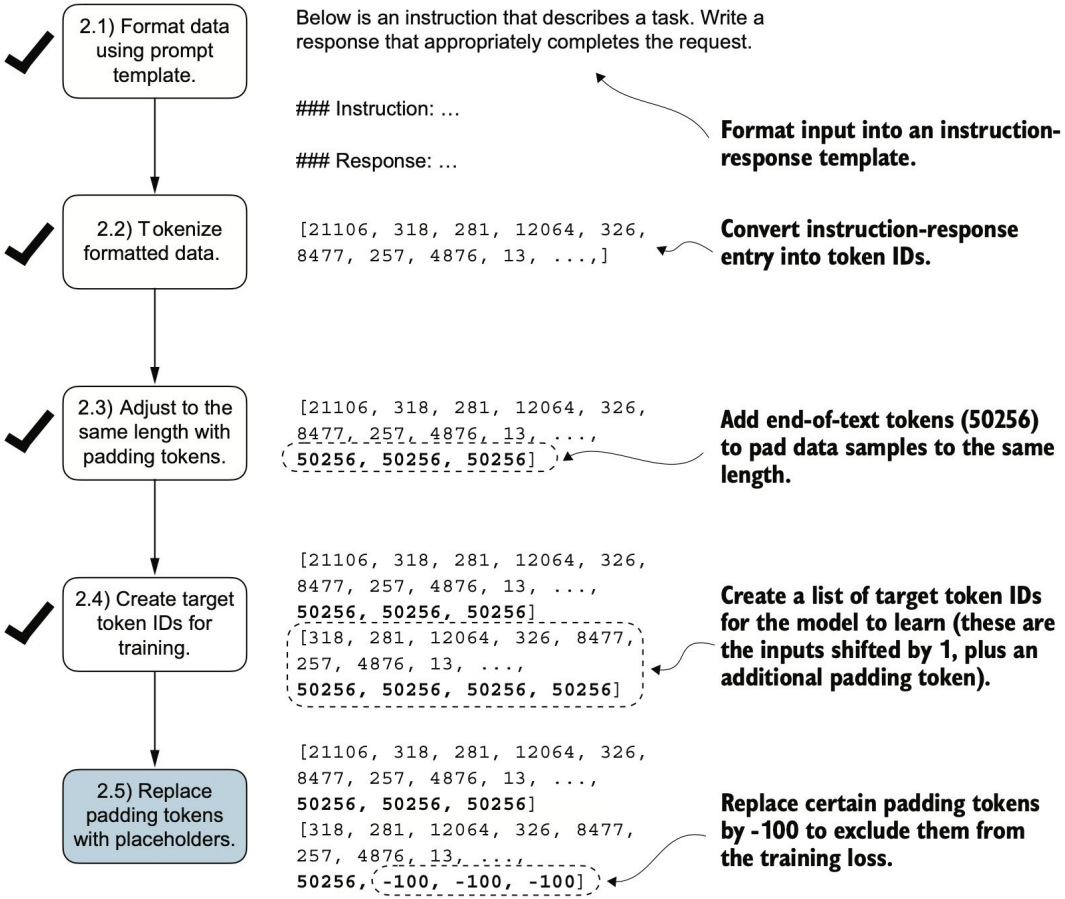


Figure 7.11 The five substeps involved in implementing the batching process. After creating the target sequence by shifting token IDs one position to the right and appending an end-of-text token, in step 2.5, we replace the end-of-text padding tokens with a placeholder value (`-100`).

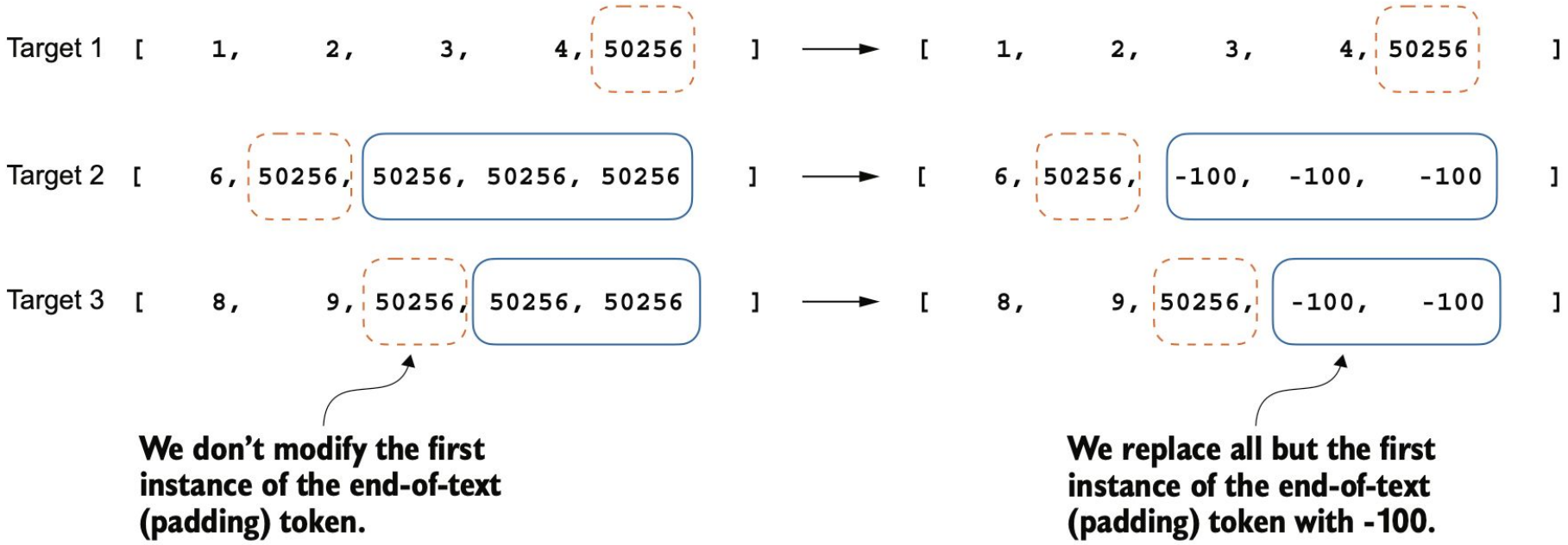


Figure 7.12 Step 2.4 in the token replacement process in the target batch for the training data preparation. We replace all but the first instance of the end-of-text token, which we use as padding, with the placeholder value `-100`, while keeping the initial end-of-text token in each target sequence.

Listing 7.5 Implementing a custom batch collate function

```
def custom_collate_fn(
    batch,
    pad_token_id=50256,
    ignore_index=-100,
```

```
tensor([[    0,     1,     2,     3,     4],
        [    5,     6, 50256, 50256, 50256],
        [    7,     8,     9, 50256, 50256]])
```

```
    (batch_max_length - len(new_item))
)
inputs = torch.tensor(padded[:-1])
targets = torch.tensor(padded[1:])
```

Truncates the last token for inputs

Shifts +1 to the right for targets

```
tensor([[    1,     2,     3,     4, 50256],
        [    6, 50256,  -100,  -100,  -100],
        [    8,     9, 50256,  -100,  -100]])
```

```
inputs_tensor = torch.stack(inputs_lst).to(device)
targets_tensor = torch.stack(targets_lst).to(device)
return inputs_tensor, targets_tensor
```

So what's so special about `-100` that it's ignored by the cross entropy loss? The default setting of the cross entropy function in PyTorch is `cross_entropy(..., ignore_index=-100)`. This means that it ignores targets labeled with `-100`. We take advantage of this `ignore_index` to ignore the additional end-of-text (padding) tokens that we used to pad the training examples to have the same length in each batch. However, we want to keep one `50256` (end-of-text) token ID in the targets because it helps the LLM to learn to generate end-of-text tokens, which we can use as an indicator that a response is complete.

In addition to masking out padding tokens, it is also common to mask out the target token IDs that correspond to the instruction, as illustrated in figure 7.13. By masking out the LLM's target token IDs corresponding to the instruction, the cross entropy loss is only computed for the generated response target IDs. Thus, the model is trained to focus on generating accurate responses rather than memorizing instructions, which can help reduce overfitting.

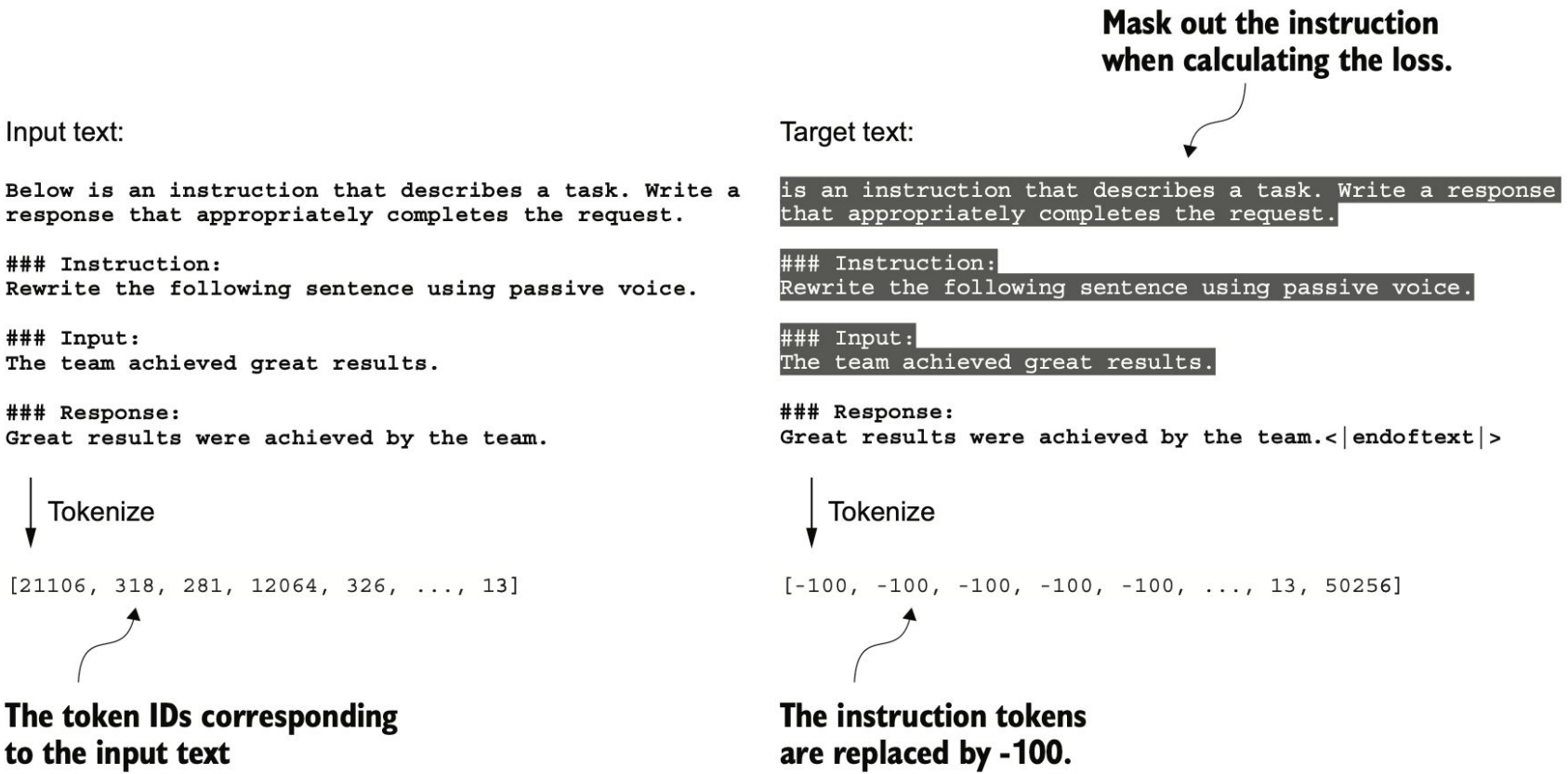


Figure 7.13 Left: The formatted input text we tokenize and then feed to the LLM during training. Right: The target text we prepare for the LLM where we can optionally mask out the instruction section, which means replacing the corresponding token IDs with the `-100` `ignore_index` value.

Mask or not to mask?

There is no consensus on whether masking the instructions is universally beneficial during instruction fine-tuning.

In Shi et al., "*Instruction Tuning With Loss Over Instructions*" arxiv.org/abs/2405.14394

> not masking the instructions benefits the LLM performance

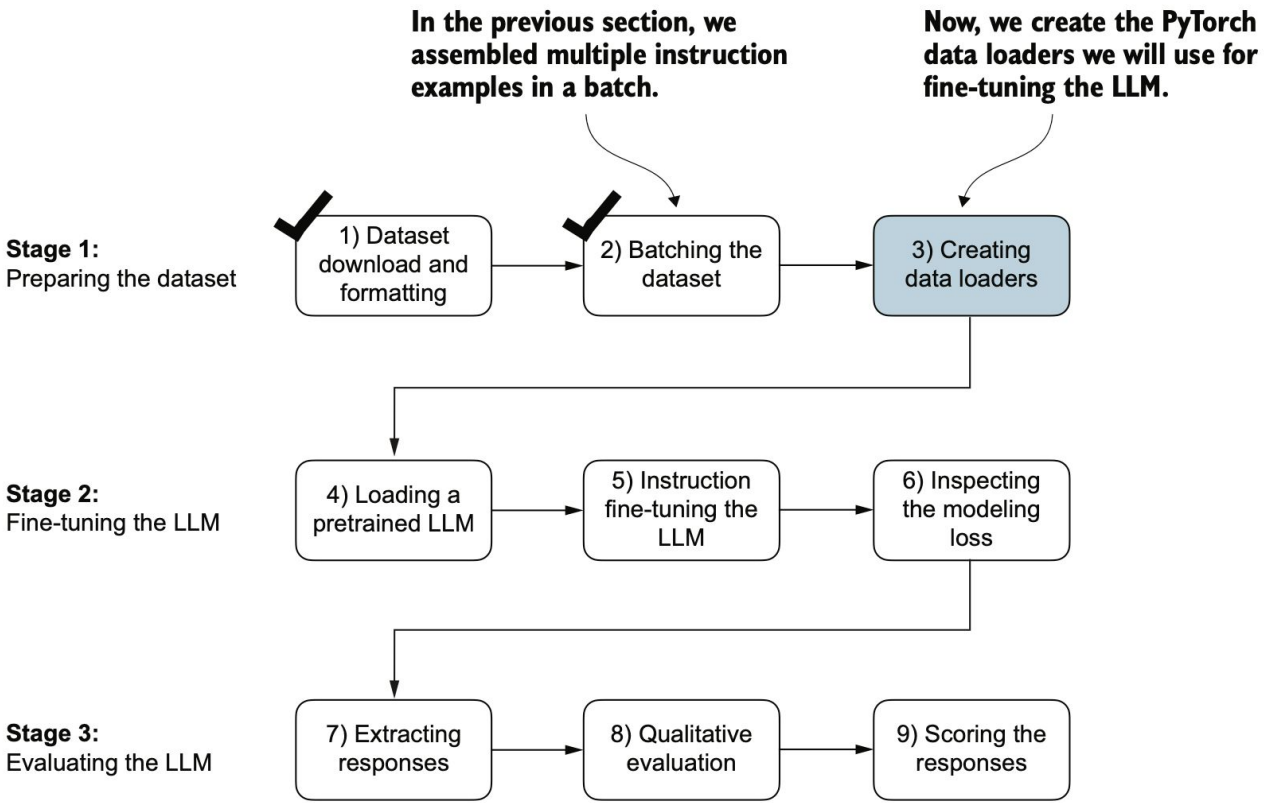


Figure 7.14 The three-stage process for instruction fine-tuning an LLM. Thus far, we have prepared the dataset and implemented a custom collate function to batch the instruction dataset. Now, we can create and apply the data loaders to the training, validation, and test sets needed for the LLM instruction fine-tuning and evaluation.

Let's examine the dimensions of the input and target batches generated by the training loader:

```
print("Train loader:")
for inputs, targets in train_loader:
    print(inputs.shape, targets.shape)
```

The output is as follows (truncated to conserve space):

```
Train loader:
torch.Size([8, 61]) torch.Size([8, 61])
torch.Size([8, 76]) torch.Size([8, 76])
torch.Size([8, 73]) torch.Size([8, 73])
...
```

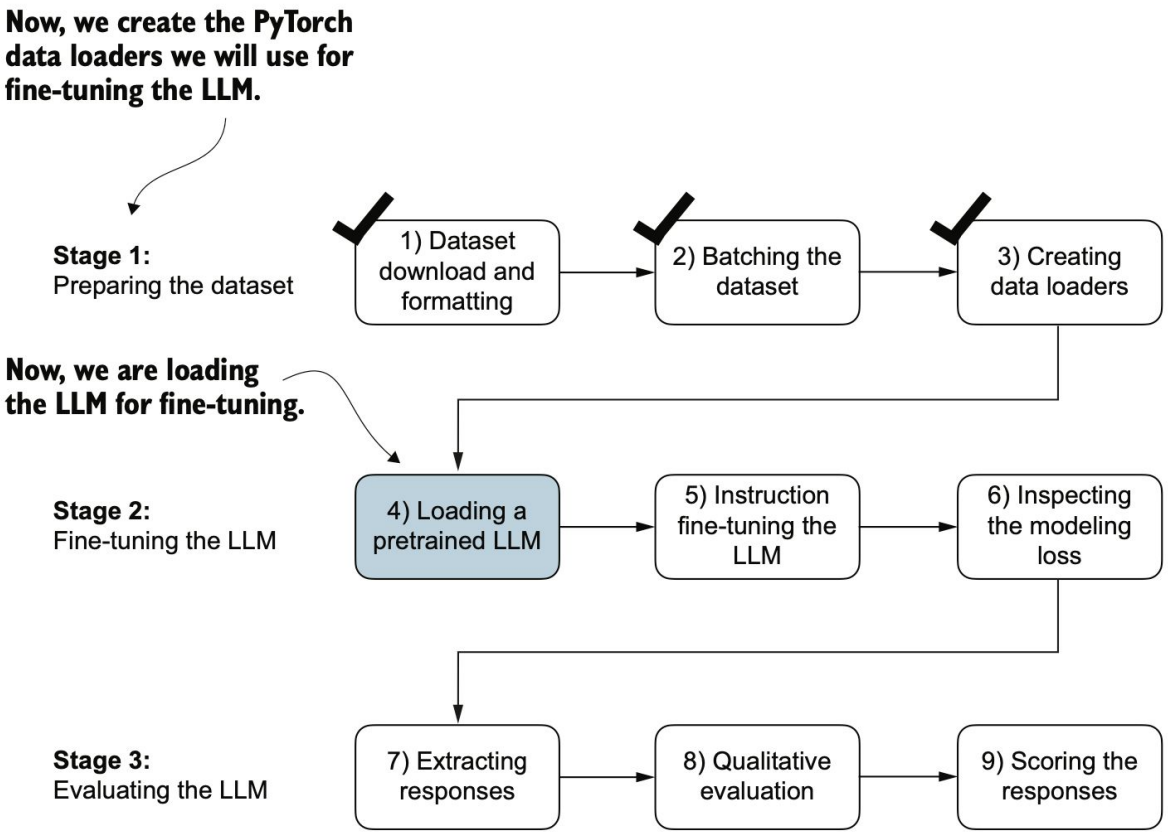


Figure 7.15 The three-stage process for instruction fine-tuning an LLM. After the dataset preparation, the process of fine-tuning an LLM for instruction-following begins with loading a pretrained LLM, which serves as the foundation for subsequent training.

Listing 7.7 Loading the pretrained model

```
from gpt_download import download_and_load_gpt2
from chapter04 import GPTModel
from chapter05 import load_weights_into_gpt

BASE_CONFIG = {
    "vocab_size": 50257,     # Vocabulary size
    "context_length": 1024,  # Context length
    "drop_rate": 0.0,        # Dropout rate
    "qkv_bias": True         # Query-key-value bias
}

model_configs = {
    "gpt2-small (124M)": {"emb_dim": 768, "n_layers": 12, "n_heads": 12},
    "gpt2-medium (355M)": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large (774M)": {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl (1558M)": {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}

CHOOSE_MODEL = "gpt2-medium (355M)"
BASE_CONFIG.update(model_configs[CHOOSE_MODEL])

model_size = CHOOSE_MODEL.split(" ")[-1].lstrip("(").rstrip(")")

settings, params = download_and_load_gpt2(
    model_size=model_size,
    models_dir="gpt2"
)

model = GPTModel(BASE_CONFIG)
load_weights_into_gpt(model, params)
model.eval();
```

Assessing the pretrained LLM's performance

```
torch.manua
input_text
print(input

The conten

Below is a
appropriat

### Instru
Convert the
```

```
### Response:

The chef cooks the meal every day.

### Instruction:

Convert the active sentence to passive: 'The chef cooks the
```

```
from chapter05 import generate, text_to_token_ids, token_ids_to_text

token_ids = generate(
    model=model,
    idx=text_to_token_ids(input_text, tokenizer),
    max_new_tokens=35,
    context_size=BASE_CONFIG["context_length"],
    eos_id=50256,
)
generated_text = token_ids_to_text(token_ids, tokenizer)
```

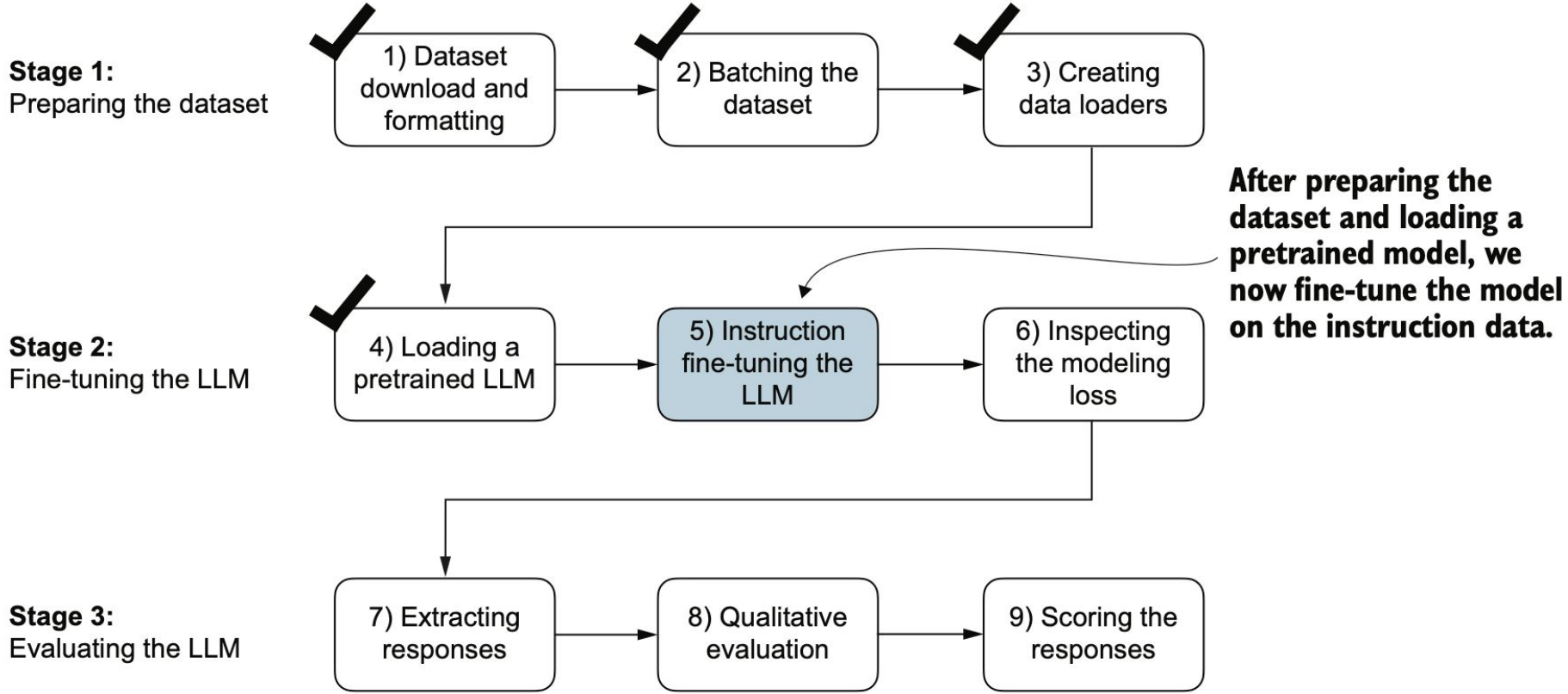


Figure 7.16 The three-stage process for instruction fine-tuning an LLM. In step 5, we train the pretrained model we previously loaded on the instruction dataset we prepared earlier.

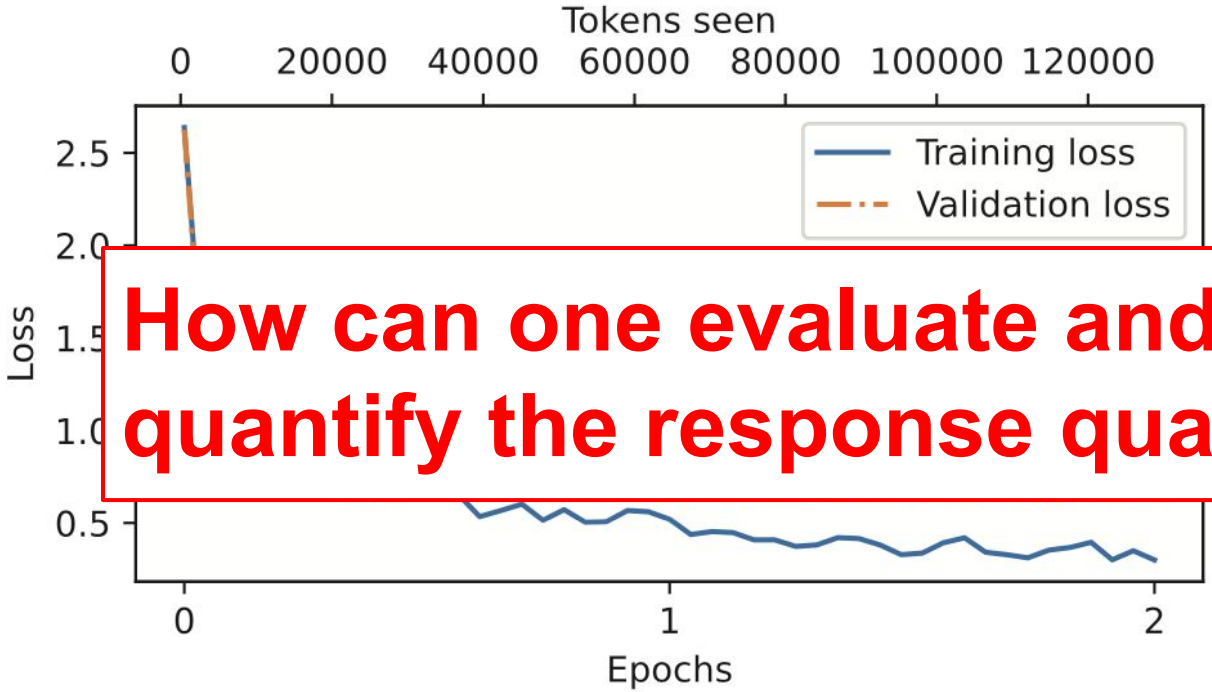
Listing 7.8 Instruction fine-tuning the pretrained LLM

```
import time

start_time = time.time()
torch.manual_seed(123)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=0.00005, weight_decay=0.1
)
num_epochs = 2

train_losses, val_losses, tokens_seen = train_model_simple(
    model, train_loader, val_loader, optimizer, device,
    num_epochs=num_epochs, eval_freq=5, eval_iter=5,
    start_context=format_input(val_data[0]), tokenizer=tokenizer
)

end_time = time.time()
execution_time_minutes = (end_time - start_time) / 60
print(f"Training completed in {execution_time_minutes:.2f} minutes.")
```

```
Ep 1 (Step 000115): Train loss 0.520, Val loss 0.665
Below is an instruction that describes a task. Write a response that
appropriately completes the request.  ### Instruction: Convert the
active sentence to passive: 'The chef cooks the meal every day.'
### Response: The meal is prepared every day by the chef.<|endoftext|>
The following is an instruction that describes a task.
Write a response that appropriately completes the request.
### Instruction: Convert the active sentence to passive:
Ep 2 (Step 000120): Train loss 0.438, Val loss 0.670
Ep 2 (Step 000125): Train loss 0.453, Val loss 0.685
Ep 2 (Step 000130): Train loss 0.448, Val loss 0.681
Ep 2 (Step 000135): Train loss 0.408, Val loss 0.677
...
Ep 2 (Step 000230): Train loss 0.300, Val loss 0.657
Below is an instruction that describes a task. Write a response
that appropriately completes the request.  ### Instruction:
Convert the active sentence to passive: 'The chef cooks the meal
every day.'  ### Response: The meal is cooked every day by the
chef.<|endoftext|>The following is an instruction that describes
a task. Write a response that appropriately completes the request.
### Instruction: What is the capital of the United Kingdom
Training completed in 0.87 minutes.
```

The training output shows that the model is learning effectively, as we can tell based on the consistently decreasing training and validation loss values over the two epochs. This result suggests that the model is gradually improving its ability to understand and follow the provided instructions. (Since the model demonstrated effective learning within these two epochs, extending the training to a third epoch or more is not essential and may even be counterproductive as it could lead to increased overfitting.)

Moreover, the generated responses at the end of each epoch let us inspect the model's progress in correctly executing the given task in the validation set example. In this case, the model successfully converts the active sentence `"The chef cooks the meal every day."` into its passive voice counterpart: `"The meal is cooked every day by the chef."`



How can one evaluate and quantify the response quality?

Figure 7.17 The training and validation loss trends over two epochs. The solid line represents the training loss, showing a sharp decrease before stabilizing, while the dotted line represents the validation loss, which follows a similar pattern.

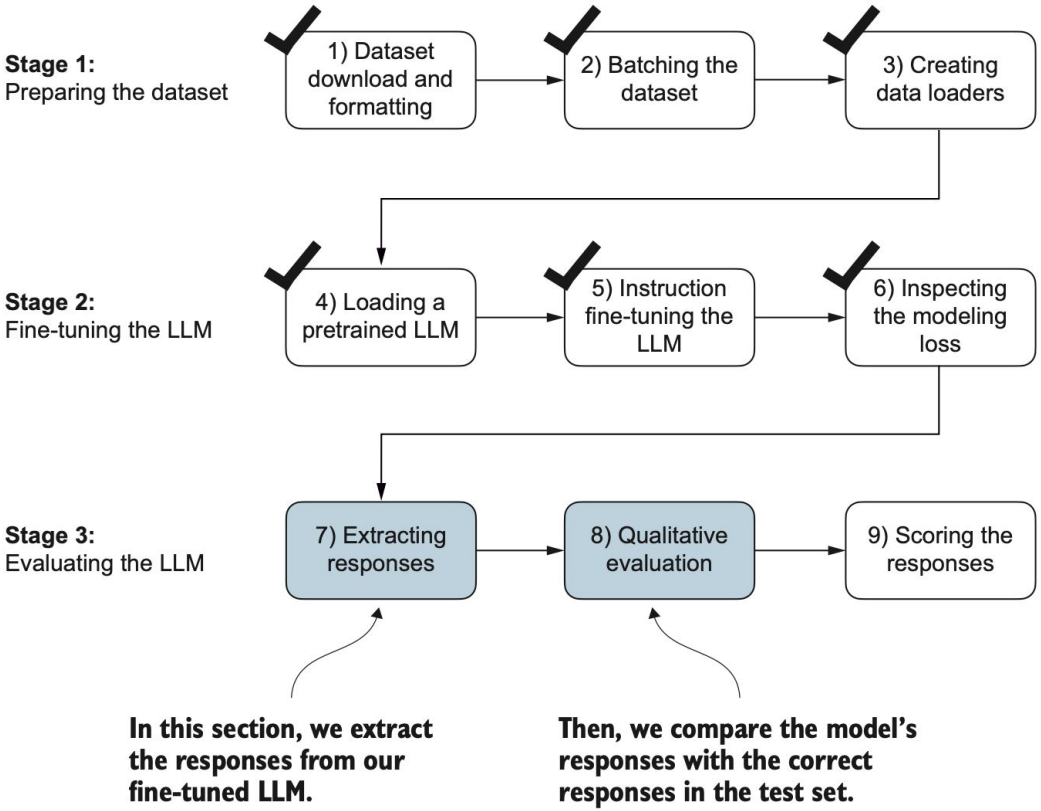


Figure 7.18 The three-stage process for instruction fine-tuning the LLM. In the first two steps of stage 3, we extract and collect the model responses on the held-out test dataset for further analysis and then evaluate the model to quantify the performance of the instruction-fine-tuned LLM.

Instruction:

Rewrite the sentence using a simile.

Input:

The car is very fast.

Correct response:

>> The car is as fast as lightning.

Model response:

>> The car is as fast as a bullet.

Instruction:

What type of cloud is typically associated with thunderstorms?

Correct response:

>> The type of cloud typically associated with thunderstorms is cumulonimbus.

Model response:

>> The type of cloud associated with thunderstorms is a cumulus cloud.

Instruction:

Name the author of ‘Pride and Prejudice.’

Correct response:

>> Jane Austen.

Model response:

>> The author of ‘Pride and Prejudice’ is Jane Austen.

Model evaluation is harder than completion fine-tuning

Short-answer and **multiple-choice benchmarks**, such as Measuring Massive Multitask Language Understanding (**MMLU**; https://arxiv.org/abs/2009.03300), which test the general knowledge of a model.

Human preference comparison to **other LLMs**, such as LMSYS chatbot arena (https://arena.lmsys.org).

Automated conversational benchmarks, where another LLM like GPT-4 is used to evaluate the responses, such as AlpacaEval (https://tatsu-lab.github.io/ alpaca_eval/).

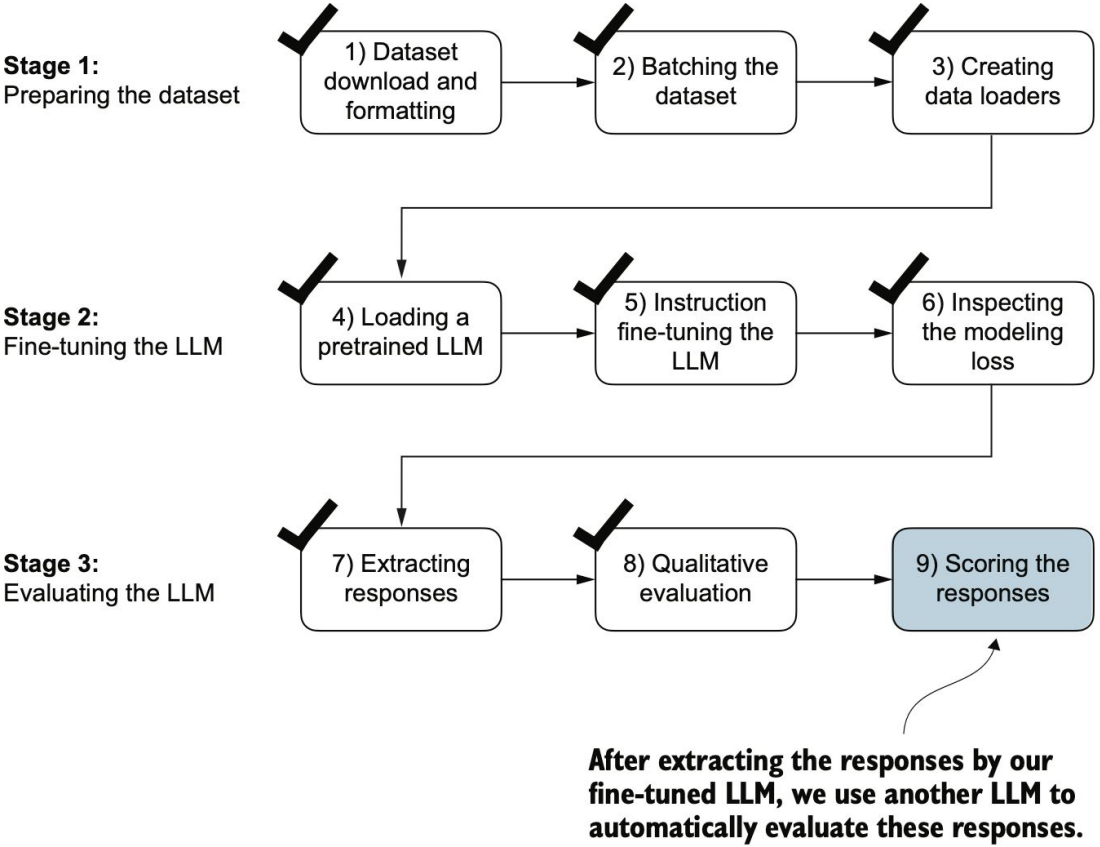


Figure 7.19 The three-stage process for instruction fine-tuning the LLM. In this last step of the instruction-fine-tuning pipeline, we implement a method to quantify the performance of the fine-tuned model by scoring the responses it generated for the test.

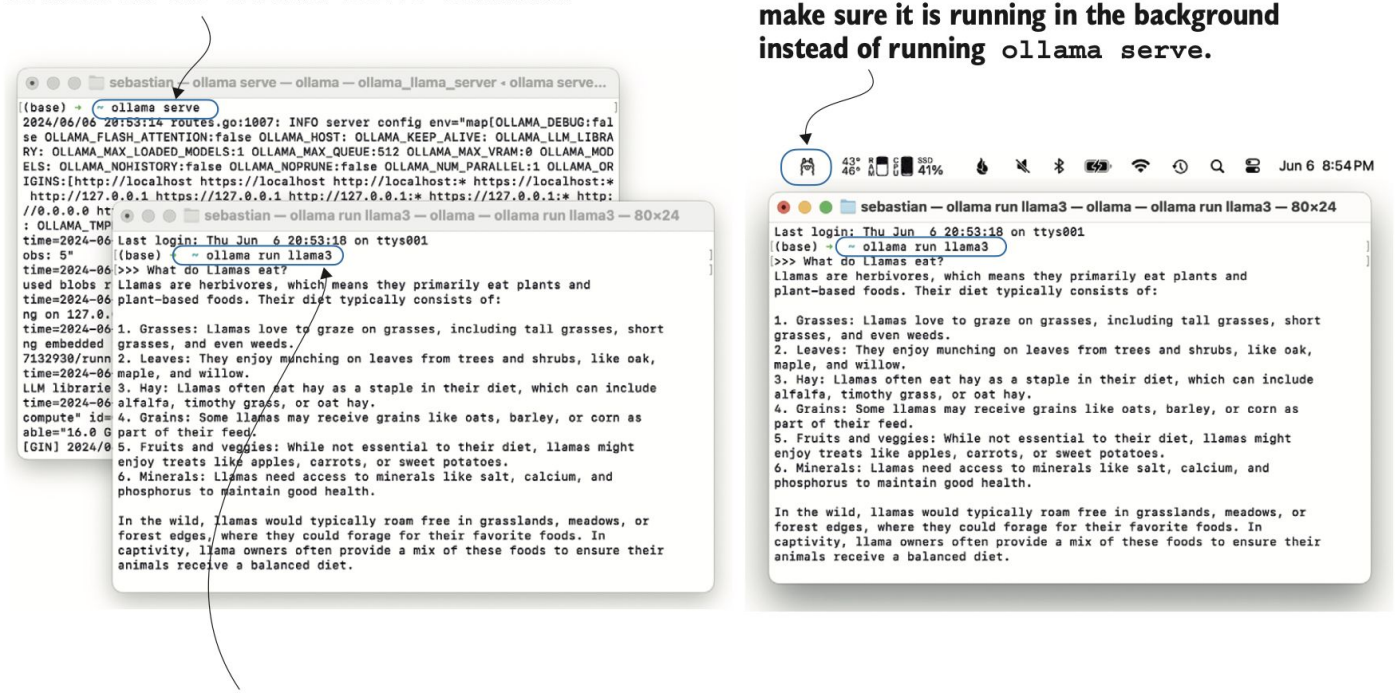


Figure 7.20 Two options for running Ollama. The left panel illustrates starting Ollama using `ollama serve`. The right panel shows a second option in macOS, running the Ollama application in the background instead of using the `ollama serve` command to start the application.

```
for entry in test_data[:3]:
    prompt = (
        f"Given the input `{format_input(entry)}` "
        f"and correct output `{entry['output']}`, "
        f"score the model response `{entry['model_response']}`"
        f" on a scale from 0 to 100, where 100 is the best score. "
    )
    print("\nDataset response:")
    print(">>", entry['output'])
    print("\nModel response:")
    print(">>", entry["model_response"])
    print("\nScore:")
    print(">>", query_model(prompt))
    print("\n-------------------------")
```

Dataset response:

>> The car is as fast as lightning.

Model response:

>> The car is as fast as a bullet.

Score:

>> I'd rate the model response "The car is as fast as a bullet." an 85 out of 100.

Here's why:

The response uses a simile correctly, comparing the speed of the car to something else (in this case, a bullet).

The comparison is relevant and makes sense, as bullets are known for their high velocity.

The phrase "as fast as" is used correctly to introduce the simile.

The only reason I wouldn't give it a perfect score is that some people might find the comparison slightly less vivid or evocative than others. For example, comparing something to lightning (as in the original response) can be more dramatic and attention grabbing. However, "as fast as a bullet" is still a strong and effective simile that effectively conveys the idea of the car's speed.

Overall, I think the model did a great job!

Dataset response:

>> The type of cloud typically associated with thunderstorms is cumulonimbus.

Model response:

>> The type of cloud associated with thunderstorms is a cumulus cloud.

Score:

>> I'd score this model response as 40 out of 100.

Here's why:

The model correctly identifies that thunderstorms are related to clouds (correctly identifying the type of phenomenon).

However, it incorrectly specifies the type of cloud associated with thunderstorms. Cumulus clouds are not typically associated with thunderstorms; cumulonimbus clouds are.

The response lacks precision and accuracy in its description.

Overall, while the model attempts to address the instruction, it provides an incorrect answer, which is a significant error.

Dataset response:

>> Jane Austen.

Model response:

>> The author of ‘Pride and Prejudice’ is Jane Austen.

Score:

>> I’d rate my own response as 95 out of 100. Here’s why:

The response accurately answers the question by naming the author of *Pride and Prejudice* as Jane Austen.

The response is concise and clear, making it easy to understand.

There are no grammatical errors or ambiguities that could lead to confusion.

The only reason I wouldn’t give myself a perfect score is that the response is slightly redundant—it’s not necessary to rephrase the question in the answer. A more concise response would be simply “Jane Austen.”

Listing 7.11 Evaluating the instruction fine-tuning LLM

```
def generate_model_scores(json_data, json_key, model="llama3"):
    scores = []
    for entry in tqdm(json_data, desc="Scoring entries"):
        prompt = (
            f"Given the input `{format_input(entry)}` "
            f"and correct output `{entry['output']}`, "
            f"score the model response `{entry[json_key]}`"
            f" on a scale from 0 to 100, where 100 is the best score. "
            f"Respond with the integer number only."
        )
        score = query_model(prompt, model)
        try:
            scores.append(int(score))
        except ValueError:
            print(f"Could not convert score: {score}")
            continue

    return scores
```

Modified instruction line to only return the score

Let's now apply the `generate_model_scores` function to the entire `test_data` set, which takes about 1 minute on a M3 Macbook Air:

```
scores = generate_model_scores(test_data, "model_response")
print(f"Number of scores: {len(scores)} of {len(test_data)}")
print(f"Average score: {sum(scores)/len(scores):.2f}\n")
```

The results are as follows:

```
Scoring entries: 100%|████████████████████████| 110/110
[01:10<00:00,  1.56it/s]
Number of scores: 110 of 110
Average score: 50.32
```

The evaluation output shows that our fine-tuned model achieves an average score above 50, which provides a useful benchmark for comparison against other models

To further improve our model's performance, we can explore various strategies, such as

- Adjusting the hyperparameters during fine-tuning, such as the learning rate, batch size, or number of epochs
- Increasing the size of the training dataset or diversifying the examples to cover a broader range of topics and styles
- Experimenting with different prompts or instruction formats to guide the model's responses more effectively
- Using a larger pretrained model, which may have greater capacity to capture complex patterns and generate more accurate responses

NOTE For reference, when using the methodology described herein, the Llama 3 8B base model, without any fine-tuning, achieves an average score of 58.51 on the test set. The Llama 3 8B instruct model, which has been fine-tuned on a general instruction-following dataset, achieves an impressive average score of 82.6.

Other Data and Models

Multi-modality instruction fine-tuning datasets

Multi-modality Instruction Fine-tuning Dataset	Modalities		# Tasks
	Modality Pair	# Instance	
MUL-TIINSTRUCT (Xu et al., 2022)[1]	Image-Text	5k to 5M per task	62
PMC-VQA (Zhang et al., 2023c)[2]	Image-Text	227k	2
LAMM (Yin et al., 2023)[3]	Image-Text	186k	9
	Point Cloud-Text	10k	3
Vision-Flan (Xu et al., 2024b)[4]	Multi-Pairs	Over 1M	200+
ALLAVA (Chen et al., 2024a)[5]	Image-Text	1.4M	2
ShareGPT4V (Chen et al., 2023a)[6]	Image-Text	1.2M	2

"Instruction Tuning for Large Language Models- A Survey".

Multi-modality instruction fine-tuned LLMs

Multi-modality Instruction Fine-tuned LLMs	# Params	Modality	Base Model		Fine-tuning Trainset	
			Model Name	# Params	Self-build	Size
InstructPix2Pix (Brooks et al., 2022)[1]	983M	I/T	Stable Diffusion	983M	Yes	450K
LLaVA (Liu et al., 2023b)[2]	13B	I/T	CLIP (Radford et al., 2021) LLaMA (Touvron et al., 2023a) LLaMA (Touvron et al., 2023a)	400M 7B 7B	Yes	158K
Video-LLaMA (Zhang et al., 2023b)[3]	-	I/T/V/A	BLIP-2 (Li et al., 2023d) ImageBind (Girdhar et al., 2023) Vicuna (Chiang et al., 2023)	- - 7B/13B	No	-
InstructBLIP (1.2B) (Dai et al., 2023)[4]	-	I/T/V	BLIP-2 (Li et al., 2023d)	-	No	-
Otter (Li et al., 2023b)[5]	-	I/T/V	OpenFlamingo (Awadalla et al., 2023)	9B	Yes	2.8M
MultiModal-GPT (Gong et al., 2023)[6]	-	I/T/V	OpenFlamingo (Awadalla et al., 2023)	9B	No	-

"Instruction Tuning for Large Language Models- A Survey".

Domain-specific instruction fine-tuned LLMs

Domain Type	Domain-specific Instruction Fine-tuned LLMs	Base Model		Trainset Size
		Model Name	# Params	
Dialogue	InstructDial (Gupta et al., 2022)[1]	T0 (Sanh et al., 2021)	3B	-
Classification	LINGUIST (Rosenbaum et al., 2022)	AlexaTM (Soltan et al., 2022)	5B	13K
Information extraction	InstructUIE (Wang et al., 2023d)[2]	FlanT5 (Chung et al., 2022)	11B	1.0M
Sentiment analysis	IT-MTL (Varia et al., 2022)[3]	T5 (Raffel et al., 2019)	220M	-
Writing	Writing-Alpaca-7B (Zhang et al., 2023d)[4]	LLaMA (Touvron et al., 2023a)	7B	-
	CoEdIT (Raheja et al., 2023)[5]	FlanT5 (Chung et al., 2022)	11B	
	CoPoet (Chakrabarty et al., 2022)[6]	T5 (Raffel et al., 2019)	11B	
Medical	Radiology-GPT (Liu et al., 2023c)[7]	Alpaca (Taori et al., 2023a)	7B	122K
	ChatDoctor (Li et al., 2023j)[8]	LLaMA (Touvron et al., 2023a)	7B	100K
	ChatGLM-Med (Wang et al., 2023a)[9]	ChatGLM (Du et al., 2022)	6B	-
Arithmetic	Goat (Liu and Low, 2023)[10]	LLaMA (Touvron et al., 2023a)	7B	1.0M
Code	WizardCoder (Luo et al., 2023)[11]	StarCoder (Li et al., 2023f)	15B	78K

"Instruction Tuning for Large Language Models- A Survey".

Advanced SFT

Catastrophic forgetting

Catastrophic forgetting refers to the phenomenon where a **model loses** its **ability** to **perform previously learned tasks** when it is being **fine-tuned on new tasks**.

The key idea of catastrophic **forgetting** is that **as the model learns new tasks**, it may **overwrite** what it **previously learned**, leading to a **loss** in **performance** on **earlier tasks**.

Instruction Fine-tuning (Full Parameter)

We need to update all the parameters while finetuning.

For a 7B model, we need to update 7 billion weights. For a 13 billion model, we need to update 13 billion weights.

Storing and updating these weights require a lot of **GPU memory.**

🎉 **Fun Fact:** Did you know, training GPT-4 involved **~25,000 A100 GPUs** over **~90-100 days**, costing OpenAI nearly **$100 million**!

Instruction Fine-tuning (Full Parameter)

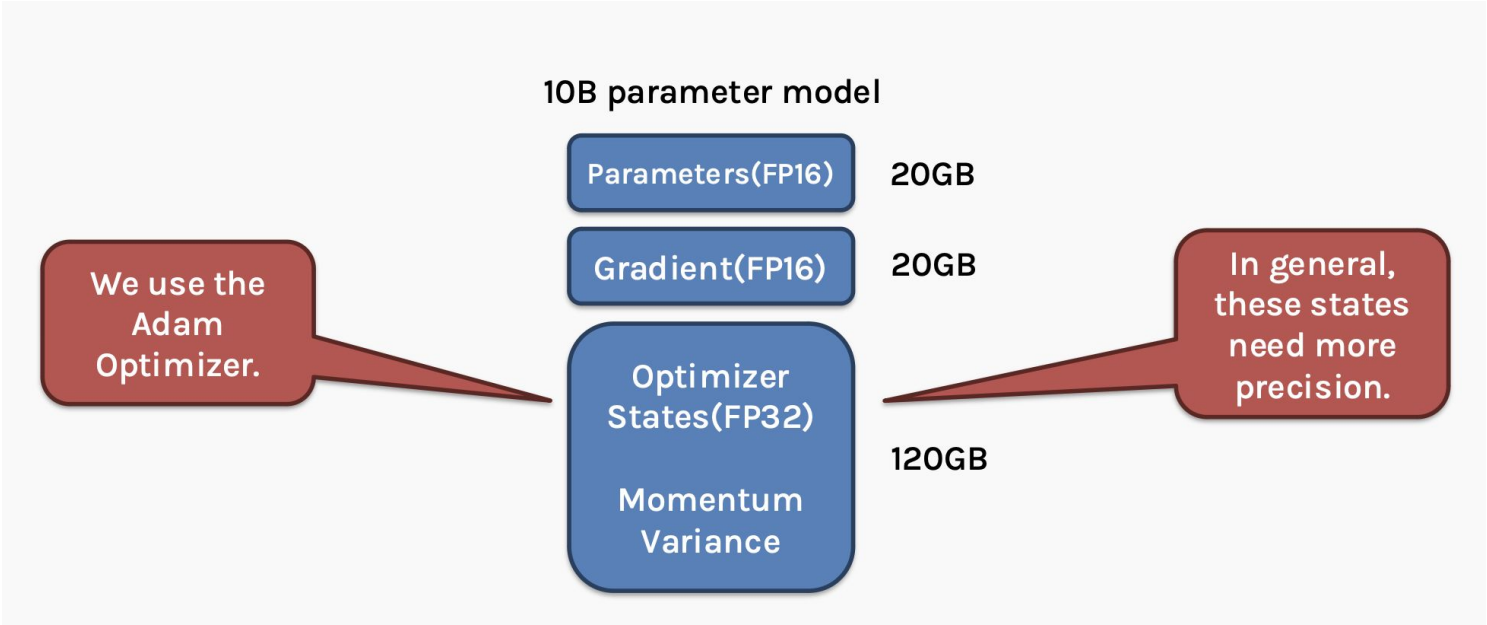
Let’s take a fine-tuning example now.

Say we want to finetune a **10 billion parameter** model. Let’s see how that looks in memory.

Assuming, we’re working with **FP16 (half precision)**, which takes approximately **2 bytes per parameter.**

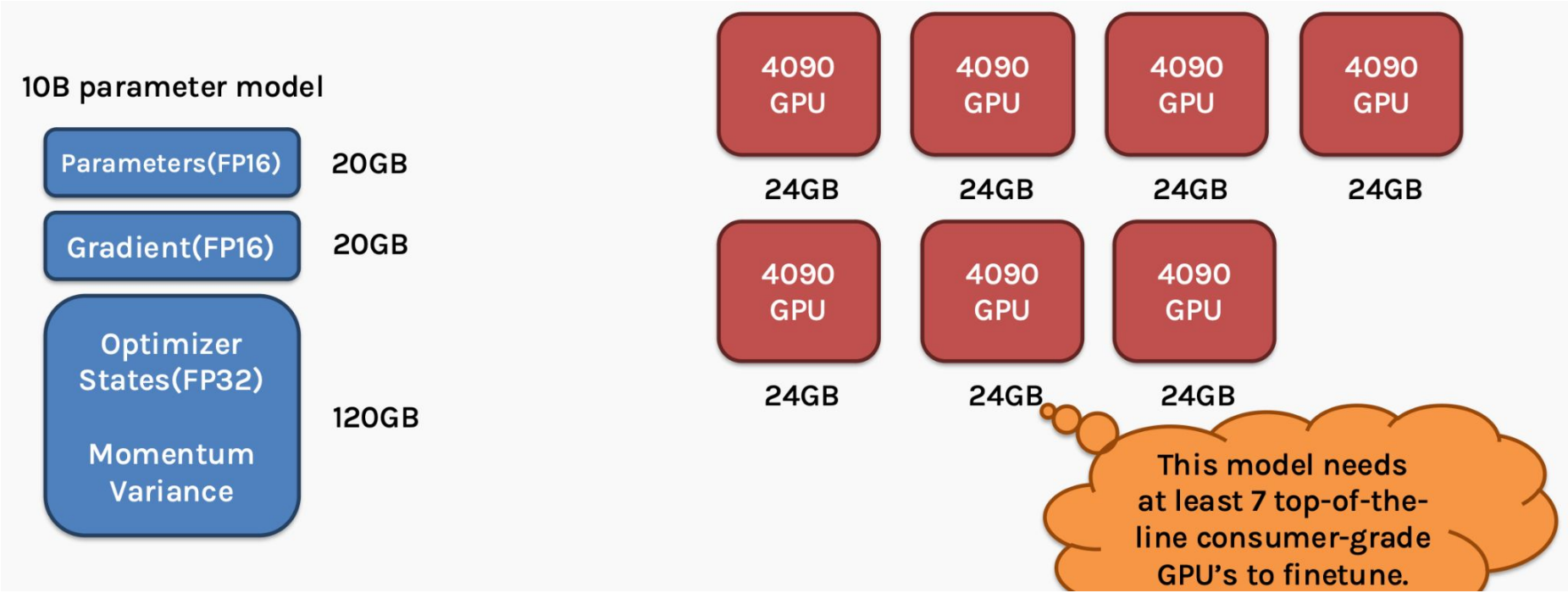
Instruction Fine-tuning (Full Parameter)

Assuming, we're working with FP16 (half precision), which takes approximately 2 bytes per parameter.



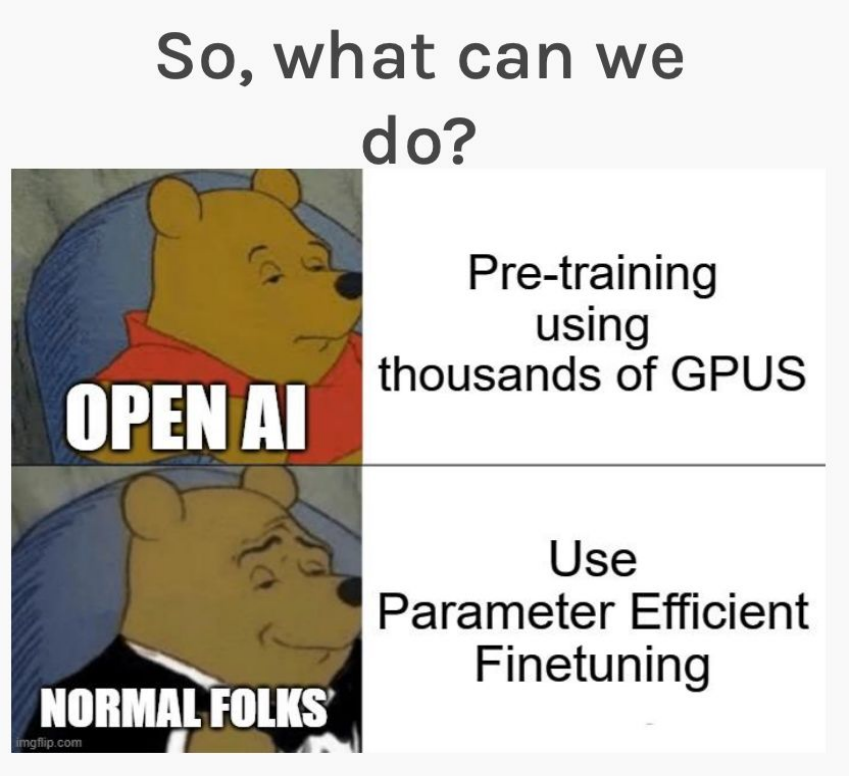
Instruction Fine-tuning (Full Parameter)

Assuming, we're working with FP16 (half precision), which takes approximately 2 bytes per parameter.



Instruction Fine-tuning (Full Parameter)

This makes full parameter fine-tuning inaccessible to normal folks like us.



Instruction-tuning (PEFT)

PEFT stands for **P**arameter **E**fficient **F**ine**t**uning.

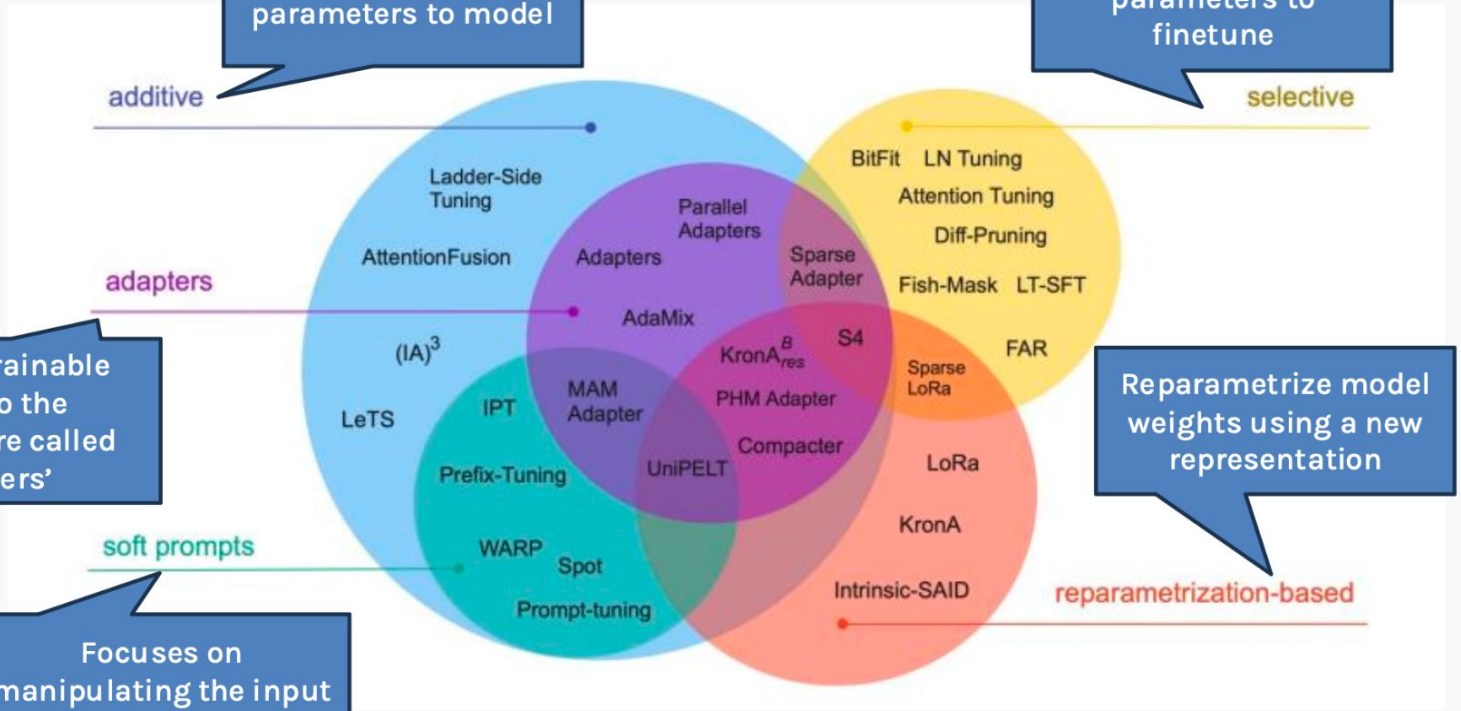
Unlike full parameter finetuning, PEFT **preserves** the vast **majority** of the model's **original weights**.

There are majorly three methods to do PEFT.

1. Additive
2. Selective
3. Reparameterization

Add trainable layers or parameters to model

Subsets the parameters to finetune



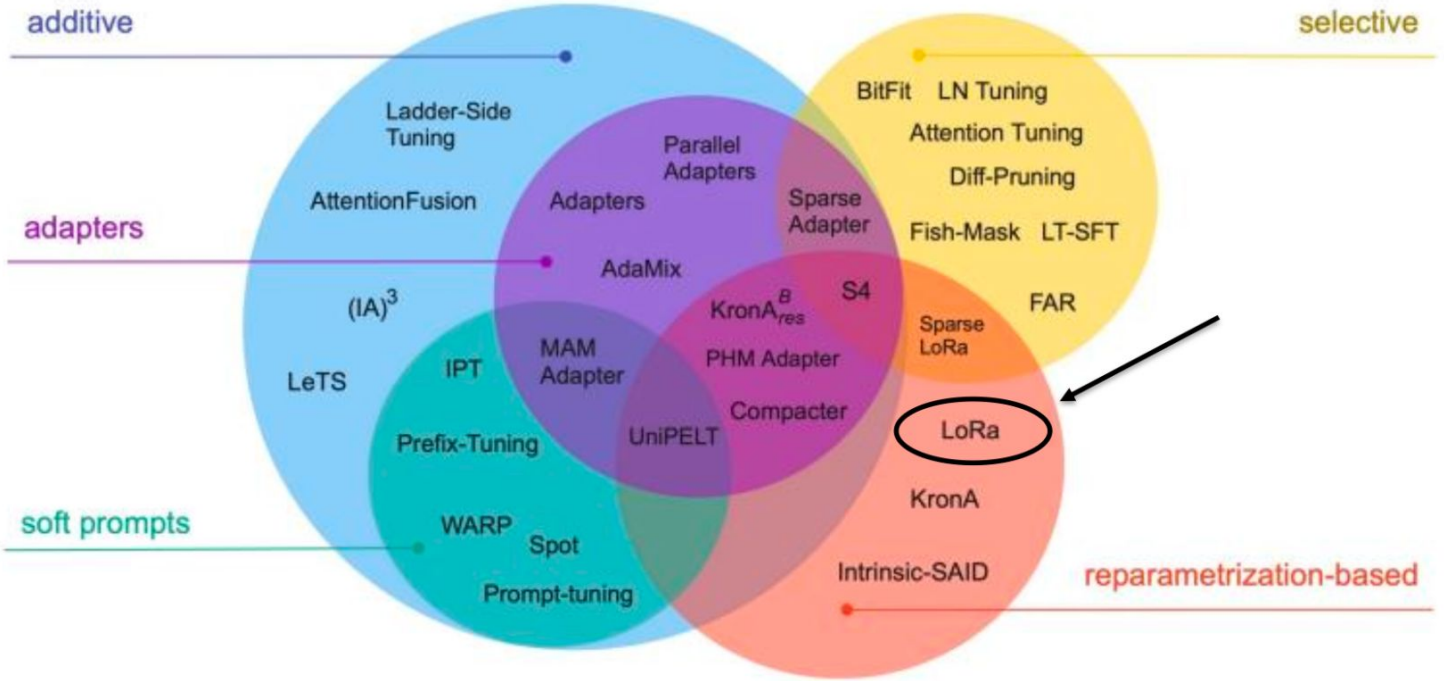
Add new trainable layers to the architecture called 'Adapters'

Reparametrize model weights using a new representation

Focuses on manipulating the input (not the same as prompt engineering)

Source: paper "Scaling Down to Scale Up" (arxiv.org)

There are a lot of techniques. We're interested in LoRA, which is one of the most popular.



Source: paper "Scaling Down to Scale Up" (arxiv.org)

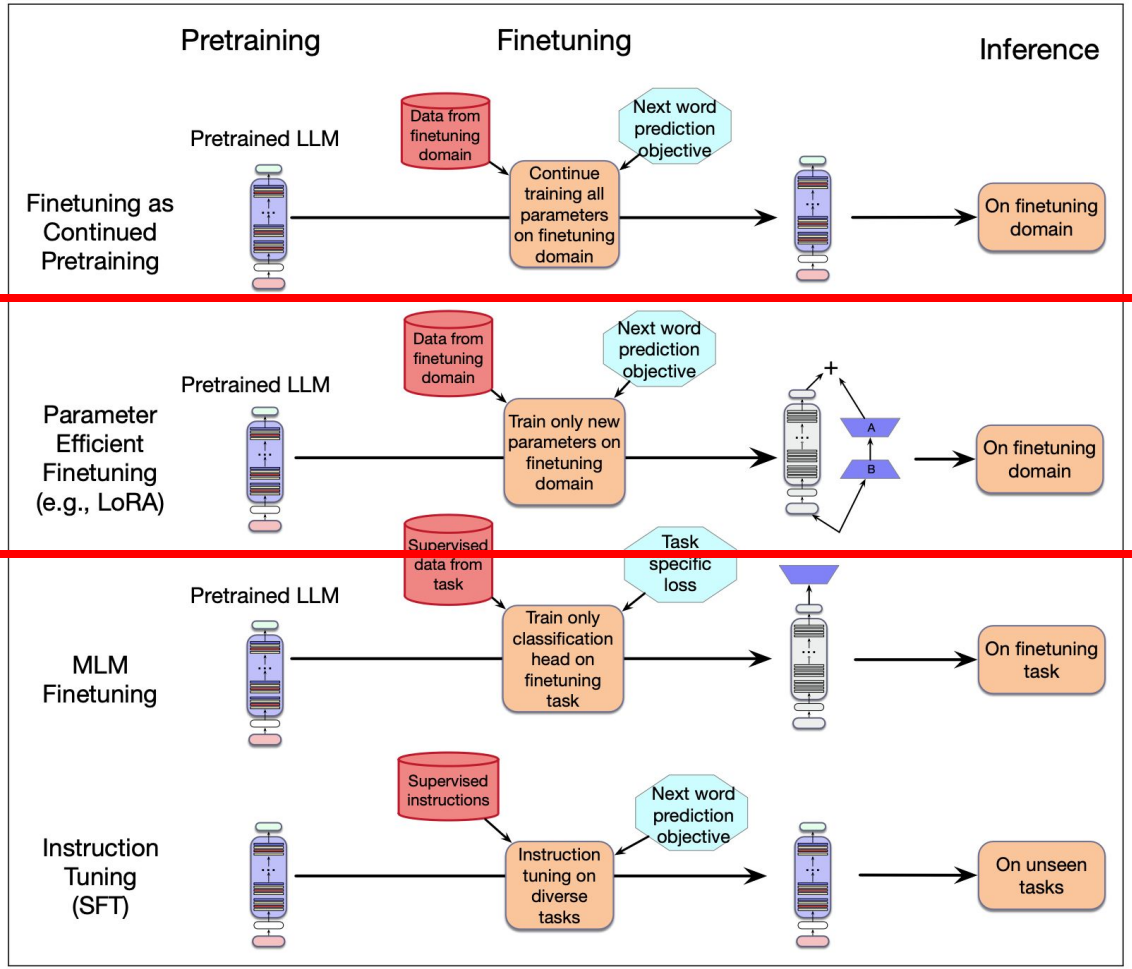


Figure 9.1 Instruction tuning compared to the other kinds of finetuning.

IMPLEMENT IT!

Summary

Where does the labels come from?

Labels often ob

Exclusive: OpenA
Less Than $2 Per H

NIAMH ROWE BUSINESS 15.10.2023 08:00 AM

Millions of Workers Are Training AI Models for Pennies

From the Philippines to Colombia, low-paid workers label training data for AI models used by the likes of Amazon, Facebook, Google, and Microsoft.

Oskarina Vero Fuentes with her dog. COURTESY OF OSKARINA VERO FUENTES

Table 6
The takeaways. We summarize some high-level suggestions for successful instruction following.

Recipes for Instruction Following
Model-related Factors (§ 7.1) • Instruction-tuned LLMs > Vanilla LLMs. • Instruction following tames LLMs to be more safe, robust, and user-friendly. • Larger LLMs benefit more from instruction following.
Instruction-related Factors (§ 7.2) • Rewriting your instruction with several epochs before it works. • Keep instruction paradigm consistent during training and testing (e.g., abstractiveness). • Design multiple instructions for one task in different wordings and perspectives. • Feeling exhausted about promoting diversity? Resort to the LLMs! • Few-shot demonstrations are useful in most cases.
Demonstration-related Factors (§ 7.3) • The choice of your few-shot examples matters a lot! • Sort your examples in a decent order. • Enhance your examples with step-by-step reasoning explanation. • Let your model exploit the input-output mapping from the examples.
Model-Instruction Alignment (§ 7.4) • Better design your instructions in a model's language (e.g., conforming to the pertaining objectives).
Data-wise Factors (§ 7.5) • Try to tune LLMs on more diverse tasks.

Lou et. al., *Large Language Model Instruction Following: A Survey of Progresses and Challenges.* https://doi.org/10.1162/coli_a_00523

Summary

The instruction-fine-tuning process **adapts a pretrained LLM** to **follow** human **instructions** and **generate** desired **responses**.

Preparing the dataset involves

- downloading an instruction-response dataset,
- formatting the entries, and
- splitting it into train, validation, and test sets.

Training batches are constructed using a *custom collate function* that **pads sequences**, **creates target token IDs**, and **masks padding tokens**.

Summary

We **load** a **pretrained GPT-2** medium **model** with **355 million** parameters to serve as the starting point for instruction fine-tuning.

The **pretrained model** is **fine-tuned** on the **instruction dataset** using a **training** loop **similar** to **pretraining**.

The Ollama application with an **8-billion-parameter Llama model** can be used to **automatically score** the **fine-tuned model's responses** on the **test set**, providing an average **score** to **quantify performance**.

Summary

Training(tuning) LMs with annotated input instructions and their output.

Pros:

Simple to implement.

Shows generalization to unseen tasks.

Cons:

It’s expensive to collect ground-truth data for tasks.

Tasks like **open-ended creative generation have no right answer.** For example: “Write me a story about a dog and her pet grasshopper.” Based on fine-tuning objectives, any deviations (even single-token) would incur a loss.

Coming next...

Preference Finetuning

Anisio Lacerda
Rodrygo Santos